Response to the Editor and Reviewers

Manuscript TACO-2026-79:

Dear Editor-in-Chief, Associate Editor, and Reviewers,

1 Overall Response

Thank you for the careful assessment of the manuscript and for the opportunity to submit a major revision. The remainder of this response letter is organized as follows. Section 2 provides a concise revision map from each reviewer comment to the corresponding change and location in the revised manuscript. Section 3 then responds to each comment in reviewer order. We append the marked revised manuscript to this response letter, with links in the Revision Map and detailed responses pointing directly to the corresponding manuscript locations. In the appended manuscript, ~~xxx~~ denotes content from the previous version that has been removed, while xxx denotes content newly added in this revision.

2 Revision Map

In the identifiers below, R$x.y$ denotes Reviewer x's y-th comment; for example, R2.3 denotes Reviewer 2, Comment 3.

For compact references below, the revised manuscript sections are abbreviated as follows: **S1** Introduction; **S2** Background and Motivation; **S3** System Design; **S4** Evaluation; **S5** Related Work; **S6** Discussion; and **S7** Conclusion. The following table maps every actionable reviewer point to the corresponding revision.

ID	Concern	Revision	Location
E §3.1	Insufficient background, weak evaluation, and unclear novelty relative to systems such as TileLang.	Expanded the background and evaluation, including stronger gains in selected inference scenarios; stated the core challenges and contributions more precisely.	**S1–S6**
R1-Q §3.2	Does "TilingInfer" enhance loop tiling?	Defined TilingInfer as inferring invariant schedule parameters, not searching for tiling strategies.	**S1**
R1.1 §3.3	Add PyTorch, ATen, and Graph IR background.	Added Graph IR, FX/ATen schemas, and constant-versus-symbolic shape background.	**S2.1**
R1.2 §3.4	Discuss benefits beyond an Ascend NPU with a lean Scalar unit.	Validated only the footprint effect of pruning a schedule-fixed library on a non-Ascend platform and bounded the latency claim to Ascend.	**S4.5**; **S6**

Continued on the next page

ID	Concern	Revision	Location
R1.3 §3.5	Separate host-control simplification from device-kernel specialization and show compiler effects beyond loop unrolling.	Clarified that host code is analyzed, not specialized; attributed gains to device kernels and documented optimizations beyond loop unrolling.	**S4.4**; **S6**
R1.4 §3.6	Compare a universal static library with per-program pruned libraries.	Measured a 542 MB universal archive against 24.3–60.7 MB per-model libraries and reported the 8–22-service break-even range.	**S4.5**
R2.1 §3.7	Parameter counts do not show which fields affect performance.	Added a five-configuration sensitivity study showing that hot-path use sites matter more than parameter count.	**S4.4**
R2.2 §3.8	Add non-dense-Transformer, model-level evaluation.	Added six end-to-end scenarios spanning DeepFM, HSTU, Llama 3.2 1B, and Qwen3.5-0.8B prefill and decode.	**S4.1**; **S4.2**
R2.3 §3.9	The standard offline-compilation baseline is weak, and the reported end-to-end gain is too small.	Enabled NPU Graph Replay, quantified scenario-dependent gains, and added Optimal as a same-kernel per-shape full-specialization oracle.	**S4.1**; **S4.2**
R2.4 §3.10	Constant propagation and partial evaluation are established techniques.	Narrowed the contribution to Graph-IR-to-C++ entry-state construction and CRVFG early-exit handling, not the compiler passes.	**S1**; **S2.3**
R3.1 §3.12	Clarify the advance over AI compilers such as TileLang and translation tools such as QiMeng-Xpiler.	Distinguished in-place specialization and pruning of expert libraries from generated or translated kernels.	**S1**; **S5**; **S6**
R3.2 §3.13	Add comparisons with AI compilers and libraries such as cuDNN.	Clarified why no direct comparison is claimed and explained the cross-implementation and cross-platform confounds.	**S1**; **S5**; **S6**
R3.3 §3.14	Explain accelerator compilation and instruction-translation differences.	Clarified that TilingInfer does not change GPU/NPU compilation or IR-to-ISA lowering; no manuscript change was needed.	No corresponding manuscript location; response only

Continued on the next page

ID	Concern	Revision	Location
R3.4 §3.15	The cross-language gap may be only simple parameter passing.	Defined the type-directed Graph-IR-to-C++ entry mapping; clarified that it is neither FFI parameter passing nor size-specific optimization.	**S1**; **S2.3**
R3.5 §3.16	What are the rules $\mathcal{R}$ in Fig. 8?	Defined $\mathcal{R}$ as five type-directed context-update rules, not tiling or size-selection rules.	**S3.2**, Fig. 8, and Table 2
R3-W §3.11	The manuscript is difficult to follow and needs clearer English.	Performed an independent full-paper language check and corrected terminology, grammar, incomplete phrases, and numerical inconsistencies.	**S1–S7**

Editorial note. To accommodate the added background and experiments within the revision page limit, we condensed material from the previous manuscript in **S2.2** and **S4.3**.

3 Detailed Responses in Reviewer Order

3.1 E: Overall assessment

Comment. The manuscript needs additional background, a stronger evaluation, and a clearer account of its core contribution.

Response. Thank you for summarizing the main concerns. We expanded the background on Graph IR, PyTorch/ATen, and expert-written operator libraries; added device-level attribution, schedule-parameter sensitivity, six end-to-end inference scenarios, and a deployment-level binary-size trade-off study; and reported more pronounced gains in the DeepFM recommendation and LLM-decode scenarios. We also revised the statement of the core challenges and contributions to define their scope and relationship more precisely. Please see our responses to R2.4 (§3.10) and R3.4 (§3.15) for the detailed contribution boundary.

Location in the revised manuscript: S1–S6.

3.2 R1-Q: Meaning of the name TilingInfer

Comment. Does TilingInfer aim to enhance loop tiling or related optimizations?

Response. “TilingInfer” refers to using constant propagation on the constant dimensions and attributes of operator shapes to infer invariant tiling data, i.e., schedule parameters, which are then used to specialize or prune kernels. The name does not imply that our system searches for or improves loop-tiling strategies.

To avoid this potential misunderstanding, we added an explanation of the name to the Introduction.

Location in the revised manuscript: S1.

3.3 R1.1: PyTorch, ATen, and Graph IR background

Comment. Please add the background needed to understand calling-context generation.

Response. We added a definition of Graph IR and used a PyTorch FX Graph as an example to explain how Graph IR represents static dimensions and runtime-dynamic dimensions. We also explained the parameter mapping between a Graph IR node and the corresponding operator schema in the underlying operator library, laying the groundwork for our subsequent design.

Location in the revised manuscript: **S2.1**, "Graph IR and External Operator Libraries."

3.4 R1.2: Applicability beyond Ascend

Comment. Please discuss performance benefits on a wider range of computing systems.

Response. On non-Ascend platforms, we validate only the footprint effect of pruning a schedule-fixed operator library, using FlashInfer on an NVIDIA GPU; we do not report non-Ascend latency gains. Performance specialization requires a schedule-generic expert-written implementation whose schedule parameters remain runtime variables. Among the open-source libraries we examined, we did not find a comparable non-Ascend implementation with the same compilation path for a controlled latency comparison. We therefore confine the cross-platform claim to the library binary-footprint reduction from per-model pruning.

Location in the revised manuscript: **S4.5**; **S6**, "Platform applicability."

3.5 R1.3: Host/device breakdown and compiler mechanisms

Comment. Separate host-side control-flow simplification from device-side specialization, and examine optimizations beyond loop unrolling.

Response. Regarding host/device separation, TilingInfer analyzes host code only to recover constant schedule parameters and kernel-launch-site reachability; it neither rewrites nor specializes the host program. We therefore attribute no performance gain to host-side control-flow simplification. We did not design host specialization because NPU Graph replay suppresses repeated launch overhead, while identical input shapes across Transformer layers allow host-computed schedule parameters to be reused. Host schedule computation is therefore not a bottleneck in our evaluated setting. Moreover, operator-library code bloat arises mainly from enumerated device-kernel template instances.

The device kernel's lower-level compiler on Ascend is closed source, and we currently have neither source access nor permission to modify it. We therefore cannot disable loop unrolling alone and must use the default optimization-pass pipeline for all compared variants.

To examine benefits beyond loop unrolling under this constraint, we conduct a controlled sensitivity analysis in **S4.4** and Fig. 15 by specializing different groups of schedule parameters. The results show benefits from simplifying control flow, address calculations, task assignment, and buffer management, not only from loop unrolling.

Location in the revised manuscript: **S4.4**; Fig. 15; **S6**.

3.6 R1.4: Universal archive versus per-model binaries

Comment. A separate custom library for every program may consume more total storage than one universal static library.

Response. We added a deployment-level FlashInfer study. The universal AOT-compiled library occupies 542 MB. Across 17 model variants, the corresponding per-model pruned libraries occupy 24.3–60.7 MB, an 88.8–95.5% reduction. Without cross-service deduplication, per-model pruning remains beneficial for up to eight model services in the worst case and up to 22 in the best case. Shared container layers, deduplication, and model mix can shift this break-even point. We therefore conclude that universal operator libraries remain attractive for highly diverse shared deployments, whereas per-model pruning is useful for isolated containers and modest model sets.

Location in the revised manuscript: **S4.5**, "Binary-Size Trade-off."

3.7 R2.1: Sensitivity to individual parameters

Comment. Counting resolved fields does not show which fields actually influence hardware stalls or performance.

Response. We agree and added a sensitivity study of the benefits of specializing scheduling parameters for the Flash Attention operator. The results are:

- Control logic, 6 scheduling parameters: 1.02×.
- Per-core tile dimensions and addressing, 14 scheduling parameters: 1.04×.
- Core-loop scheduling and memory resources, 14 scheduling parameters: 1.07×.
- The three groups jointly, 34 scheduling parameters: 1.10×.
- All constant schedule parameters specialized by TilingInfer, 352 parameters: 1.14×.

For the evaluated D-Attn implementation and shapes, specializing the 34 hot-path parameters above achieves 75.0% of TilingInfer's full gain, while the remaining 318 parameters contribute only an additional 3.4 percentage points. Within this scope, the types and use sites of the specialized schedule parameters matter more than their number.

Location in the revised manuscript: S4.4, "Sensitivity to Resolved Schedule Parameters."

3.8 R2.2: Non-dense-Transformer evaluation

Comment. Please add model-level evidence from CV or recommendation architectures.

Response. We expanded the evaluation to six representative end-to-end inference scenarios: DeepFM recommendation, HSTU generative recommendation, Llama 3.2 1B prefill and decode, and Qwen3.5-0.8B prefill and decode. For all six scenarios, we report end-to-end speedup, device-kernel category composition, and category-level specialization speedup.

Location in the revised manuscript: S4.1 and S4.2.

3.9 R2.3: Magnitude and baselines

Comment. The reported end-to-end improvement is too small, and standard offline compilation is a weak baseline.

Response. Across six representative inference scenarios, TilingInfer achieves a 1.05× geometric-mean end-to-end speedup. Gains are lower for the compute-intensive HSTU and LLM prefill scenarios, where tensor computation dominates. In contrast, DeepFM and the two LLM decode scenarios average 1.10× because their shorter, relatively memory-intensive kernels expose a larger fraction of removable scalar and scheduling overhead.

The average gain of the two decode scenarios increased from approximately 3% in the previous manuscript to 10% after we enabled NPU Graph replay for all compared methods. This suppresses repeated CPU launch overhead that previously masked the contribution of device-kernel specialization.

To address the baseline concern, we compare TilingInfer with Optimal, a per-shape full-specialization oracle that specializes all schedule parameters produced by host scheduling. Although impractical for production because it recompiles every shape, Optimal provides a same-kernel reference. TilingInfer and Optimal average 1.05× and 1.08× across all six scenarios, and 1.10× and 1.14× across the two decode scenarios, respectively. Each data point averages 90 measured runs after 10 warm-up runs to reduce measurement noise.

Location in the revised manuscript: S4.1 and S4.2.

3.10 R2.4: Classical partial evaluation and constant propagation

Comment. The work appears to integrate established compiler techniques rather than introduce a new concept.

Response. We agree that constant propagation, partial evaluation, and template instantiation are mature techniques, and we do not claim these compiler passes themselves as novel. TilingInfer's contribution is to establish two prerequisites missing when applying these techniques to expert-written operator libraries.

First, although runtime FFI can pass concrete tensor and attribute values, it does not expose the distinction between model constants and symbolic dimensions in Graph IR as compile-time facts usable by a separately compiled C++/LLVM operator library. For example, both dimensions of the Graph IR shape `[s_B, 40]` appear as integer loads through `Tensor::size(i)` after entering C++. TilingInfer uses type-directed context-update rules to materialize 40 as a constant and preserve `s_B` as unknown in the C++ entry state, without analyzing Python/C++ FFI glue code. These rules cover five core parameter types and their standard nested compositions and are not written for specific sizes.

Second, TilingInfer uses CRVFG to identify early-exit edges in shape checkers before constant propagation, preventing error paths from polluting the analysis state of normal paths while avoiding full path-sensitive analysis. Thus, the contribution is not a new partial-evaluation theory, but cross-layer entry-state construction and early-exit identification mechanisms that enable existing compiler optimizations to be applied to expert-written operator libraries. We have clarified this contribution boundary in the revised manuscript.

Location in the revised manuscript: **S1** contribution statement; **S2.3**, "Challenges and Design Insights."

3.11 R3-W: Clarity and English expression

Comment. The manuscript is difficult to follow, and the English should be refined.

Response. Thank you for this comment on the presentation. We performed an independent full-paper language check in addition to reorganizing the manuscript. The revision standardizes terminology and examples and corrects subject–verb agreement, incomplete phrases, awkward constructions, and numerical inconsistencies throughout.

Location in the revised manuscript: Substantial revisions throughout **S1**–**S7**.

3.12 R3.1: AI compilers, TileLang, and QiMeng-Xpiler

Comment. The manuscript should explain what remains distinctive relative to AI compilers and code-translation tools.

Response. These approaches share the ultimate goal of improving operator performance, but differ in their optimization targets and compilation paths. AI compilers such as TileLang start from high-level operator descriptions and generate new specialized kernels through multi-level IRs, schedule search, or autotuning. Code-translation tools such as QiMeng-Xpiler use LLM-based transformations, symbolic repair, and tuning to generate or port target implementations.

TilingInfer neither regenerates operator implementations nor searches for new scheduling strategies. Instead, it targets existing expert-written operator libraries and introduces constants from Graph IR into their C++/LLVM compilation flows to perform partial evaluation on kernels and prune unreachable precompiled instances.

Thus, TilingInfer's advance lies in providing existing expert-written operator libraries with automatic specialization and pruning capabilities across Graph IR and C++/LLVM. It takes a different but complementary technical path from the generative approaches above.

We added an explanation of these distinctions to the Introduction and Related Work.

Location in the revised manuscript: **S1**; **S5**, "AI Compilers and Kernel Generation"; **S6**.

3.13 R3.2: Comparisons with AI compilers and cuDNN

Comment. The evaluation should compare against existing AI compilers and specialized libraries such as cuDNN.

Response. We carefully considered this comment but concluded that TilingInfer should not be compared directly with AI compilers or specialized libraries such as cuDNN.

For AI compilers, as discussed in our response to R3.1, the scheduling implementations of AI-compiler-generated kernels differ from those in the expert-written operator libraries optimized by TilingInfer. A direct comparison would therefore conflate the benefits of specialization with performance differences in the kernels' own scheduling implementations. To illustrate this gap, we additionally measured TileLang-Ascend and the CANN expert-written operator library on Ascend, as shown in Fig. 1:

- Grouped Matmul: CANN averages 100.2 TFLOPS, while TileLang-Ascend averages 28.2 TFLOPS.
- Matmul: CANN averages 173.6 TFLOPS, while TileLang-Ascend averages 97.0 TFLOPS.

We did not compare against well-known specialized libraries such as cuDNN for two reasons:

(1) Our operator partial-evaluation experiments are conducted on Ascend, whereas cuDNN targets NVIDIA GPUs, making a direct cross-platform comparison difficult. Our response to R1.2 discusses in detail why the operator partial-evaluation experiments are based on Ascend.
(2) For pruning precompiled operator instances, cuDNN's core library is closed source. TilingInfer cannot obtain its LLVM IR and therefore cannot optimize cuDNN itself. A direct library-size comparison between FlashInfer and specialized libraries such as cuDNN would likewise be difficult to interpret.

Moreover, CANN and FlashInfer, the two operator libraries optimized in this work, are both expert-written libraries widely used in production. State-of-the-art inference engines provided by open-source communities or hardware vendors, including vLLM/vLLM-Ascend, SGLang/SGLang-Ascend, and MindIE, use these libraries by default.

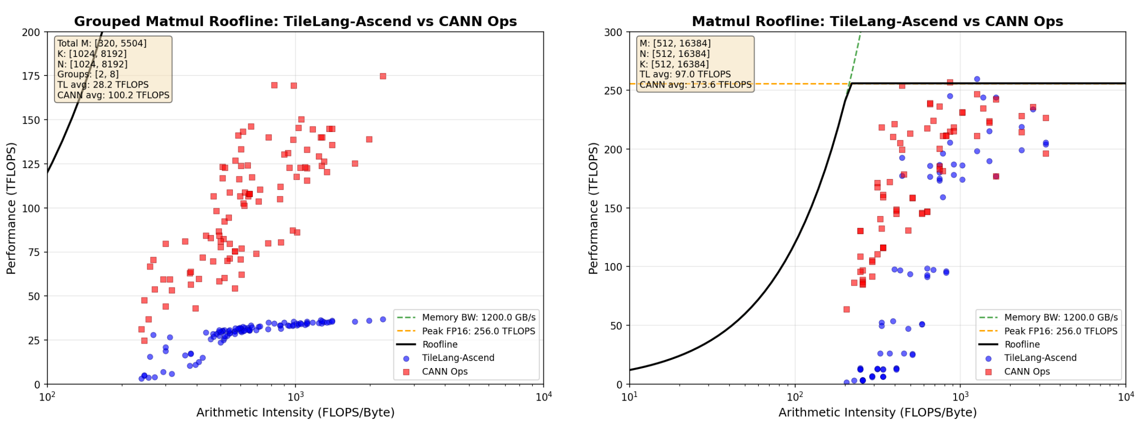


Fig. 1. Ascend roofline characterization of CANN and TileLang-Ascend for Grouped Matmul (left) and Matmul (right).

Location in the revised manuscript: **S1**; **S5**; **S6**. The roofline plots above are included only in this response and were not added to the manuscript.

3.14 R3.3: NPU and GPU compilation paths

Comment. The design should explain compilation and instruction translation for the two accelerators.

Response. TilingInfer operates within the compilation flow of existing expert-written libraries. It analyzes and transforms LLVM IR emitted from their C++ host and device implementations, but it does not modify the subsequent backend-specific lowering from compiler IR to GPU or NPU machine instructions. The existing accelerator backend and instruction-selection pipelines remain unchanged.

Because backend instruction lowering is outside TilingInfer's optimization scope and is not required to explain its cross-layer constant propagation, we did not add backend-lowering details to the manuscript.

Location in the revised manuscript: No corresponding manuscript location; response clarification only.

3.15 R3.4: Is the cross-language gap a real challenge?

Comment. Determining tensor shapes and attributes may appear to be a collection of independent size-specific optimizations followed by simple integration.

Response. Thank you for identifying the ambiguity. Inferring tensor shapes and operator attributes is not itself a contribution of this work; systems such as Dynamo and TVM Relax already provide symbolic shape inference. The challenge is to materialize the constants and symbolic information available in Graph IR into an entry state that C++/LLVM analysis can use.

Runtime FFI can pass concrete tensor-shape and operator-attribute values, but it does not preserve the distinction between constant and symbolic dimensions in Graph IR as compile-time information usable by a separately compiled operator library. After entering C++, parameters such as Tensor, List, and Optional are represented as composite objects with nested memory layouts, and their accesses are lowered to pointer arithmetic and field loads. End-to-end cross-language static analysis would also need to process framework dispatch, type conversion, object construction, value copying, reference counting, and other FFI code unrelated to the target mapping, making the analysis expensive and framework-specific.

TilingInfer defines type-directed context-update rules for the five core parameter types Scalar, String, Tensor, List, and Optional. These rules directly materialize Graph IR metadata as abstract states at the corresponding C++ object fields and memory locations, while recursive rules support standard nested compositions. The rules are selected by parameter type rather than concrete size: TilingInfer neither enumerates shapes nor implements separate optimizations for different sizes, but instead propagates the recovered constants through the same C++/LLVM implementation to kernel launch sites.

Therefore, this process is neither simple parameter passing nor a collection of independent size-specific optimizations.

Location in the revised manuscript: **S1**; **S2.3**, "Challenges and Design Insights."

3.16 R3.5: Meaning of the rules $\mathcal{R}$

Comment. What do the rules in Fig. 8 represent, and are they optimizations for different operator sizes?

Response. These rules are neither tiling rules nor size-selection rules. They are mappings from Graph IR to C++ memory objects for five basic operator parameter types: Scalar, String, Tensor, List, and Optional. For the metadata of a given Graph IR node, the rules define how to construct the C++ entry state.

To avoid reader confusion, we revised the workflow text and title in Fig. 8, introduced the meaning of the rule family $\mathcal{R}$ at the beginning of Section 3.2, and listed the symbol for each rule in Table 2 so that the role of $\mathcal{R}$ is explicit.

Location in the revised manuscript: **S3.2**, especially Fig. 8 and Table 2.

Sincerely,
on behalf of all authors

Optimizing Dynamic-Shape Operator Libraries via Automatic ~~Kernel~~ Model-Specific Specialization on AI Accelerators

~~Deep learning (DL) frameworks rely heavily on operator libraries to support diverse model architectures and dynamic input shapes. To maintain flexibility, these libraries employ generic kernels with numerous runtime parameters for optimal scheduling. However, relying on runtime resolution for these parameters hurts performance, whereas statically enumerating them for efficiency leads to severe code bloat, creating a fundamental dilemma: accepting performance degradation with generic kernels or suffering severe code bloat through exhaustive specialization.~~

Deep-learning (DL) systems rely on expert-written operator libraries for high performance across dynamic input shapes. These libraries use shape-dependent schedule parameters, such as tile sizes and on-chip buffer layouts. Schedule-generic expert-written operator libraries preserve shape coverage with a single kernel but limit compile-time optimization, whereas schedule-fixed expert-written operator libraries precompile combinations of schedule parameters and incur binary bloat. Kernel-generation compilers do not resolve this trade-off because they generate new kernels rather than specialize existing libraries.

~~A promising solution is to specialize kernels by propagating model-specific constants from the framework to the compiler. However, this approach faces two challenges: (1) Cross-language semantic gap, where constant information in the high-level Python DSL fails to propagate to device-native C++ compilers, and (2) Control flow obstruction, where extensive validity checks on dynamic dimensions generate massive execution paths handling invalid shapes, thereby impeding the precision and efficiency of constant propagation.~~

Although many schedule parameters are fixed for a given model, the corresponding model-specific constants recorded in Graph IR are absent from the LLVM IR of separately compiled C++ operator libraries. Runtime argument transmission through Python/C++ FFI does not make these values available as compile-time facts. Propagating model-specific constants from the framework enables specialization, but existing toolchains face two challenges: the Python/C++ semantic gap prevents constant information from reaching the native compiler, and shape checkers for dynamic dimensions introduce many early-exit paths that weaken constant propagation.

~~We present TilingInfer, the first framework designed to automate kernel specialization in cross-language and dynamic-shape scenarios. To bridge the semantic gap, TilingInfer employs program synthesis to generate partial evaluation programs directly from the Graph IR. To handle control flow, it constructs a Conditional Return-Value Flow Graph to trace the relationship between shape check conditions and early-exit paths, enabling efficient identification and pruning of infeasible execution paths through lightweight constant comparison.~~

We present TilingInfer, a static-analysis framework that uses LLVM to propagate Graph IR shape information through existing C++ operator libraries and derive constant schedule parameters for kernel specialization and redundant-code elimination. To address the two challenges above, TilingInfer introduces two core designs. First, type-specific context-update rules construct C++ entry states directly from Graph IR shapes and attributes, avoiding analysis of Python/C++ FFI glue code. Second, a Conditional Return-Value Flow Graph identifies early-exit paths and prevents their states from affecting normal paths, preserving precision without analyzing every path.

~~We implement TilingInfer on Ascend NPUs and NVIDIA GPUs. Evaluations on state-of-the-art libraries (CANN and FlashInfer) demonstrate its effectiveness: TilingInfer achieves an average 1.06× speedup (up to 1.17×) on Ascend~~

Author's Contact Information:

Manuscript submitted to ACM

~~operators and 1.03× speedup on LLM inference. Furthermore, it reduces the binary size of FlashInfer by over 96% on GPUs, demonstrating significant potential for library slimming.~~

On an Ascend NPU, TilingInfer achieves a 1.06× geometric-mean device-kernel speedup across 14 operators, with a maximum of 1.17×, and a 1.05× geometric-mean end-to-end speedup across six representative inference scenarios. It provides larger benefits in the DeepFM recommendation and LLM decode scenarios, which average 1.10×. On an NVIDIA GPU, it reduces a 542 MB operator-library archive by 88.8–95.5% across 17 model variants.

CCS Concepts: • **Software and its engineering** → **Dynamic compilers**; • **Computer systems organization** → *Single instruction, multiple data*; • **Computing methodologies** → *Neural networks*; • **General and reference** → *Performance.*

Additional Key Words and Phrases: compilers, static analysis, deep learning, kernel specialization, constant propagation, AI accelerators, operator libraries

ACM Reference Format:

. 2026. Optimizing Dynamic-Shape Operator Libraries via Automatic ~~Kernel~~ Model-Specific Specialization on AI Accelerators. 1, 1 (July 2026), 34 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

~~DL frameworks and serving systems~~ [1, 3, 16, 45] ~~rely heavily on hand-written operator libraries~~ [4, 7, 8, 13, 22, 40] ~~to ensure efficient operator implementation. To achieve universal applicability, these libraries must handle input dynamics arising from two main sources: (1) Cross-model heterogeneity, involving diverse algorithmic variants (e.g., attention mechanisms~~ [2, 5, 29, 36]~~) and varying attributes (e.g., head numbers~~ [9, 35, 39]~~); and (2) Input shape variability, where dimensions like batch sizes and sequence lengths vary at runtime.~~

Deep-learning (DL) frameworks and serving systems [1, 3, 16, 45] rely heavily on expert-written operator libraries [4, 7, 8, 13, 22, 40]. These libraries must cover input dynamics from both cross-model heterogeneity (such as different algorithmic variants [2, 5, 29, 36] and architectural attributes [9, 35, 39]) and runtime variation in dimensions (such as batch size and sequence length). The performance of each resulting operator configuration depends not only on its algorithm but also on its schedule parameters, including tile sizes, loop bounds, pipeline stages, and memory layout choices. Different shapes can have different optimal schedules. Consequently, when input shapes are dynamic, the operator implementation must select a high-performance schedule based on the runtime shape.

~~Existing operator libraries adopt different specialization strategies depending on the underlying hardware characteristics, yet both approaches face critical limitations as illustrated in~~ Figure 1~~.~~

Existing approaches can be classified by how their generated binaries represent schedule parameters. A schedule-fixed kernel compiles the key schedule parameters as constants, so each compiled kernel realizes a fixed schedule. A schedule-generic kernel retains the schedule parameters as runtime variables, allowing one compiled kernel to realize different schedules for different shapes. The four approaches in Figure 1 comprise three schedule-fixed approaches (①–③) and one schedule-generic approach (④).

~~Meanwhile, although AI compilers~~ [6, 30, 34, 43, 46] ~~offer an alternative by generating specialized kernels at runtime, they are typically confined to specific high-level IRs (e.g., Triton IR~~ [34] ~~and Relax~~ [17]~~) and incur non-negligible runtime compilation overhead.~~

AI compilers produce schedule-fixed kernels through schedule search and code generation. ① A static-shape AI compiler takes a concrete shape as input, searches for or autotunes a schedule for that shape, and generates a shape-specific kernel, often through a just-in-time (JIT) compilation path [32–34, 44]. An unseen shape triggers another round of schedule search and code generation. ② A dynamic-shape AI compiler instead searches for schedules over a declared

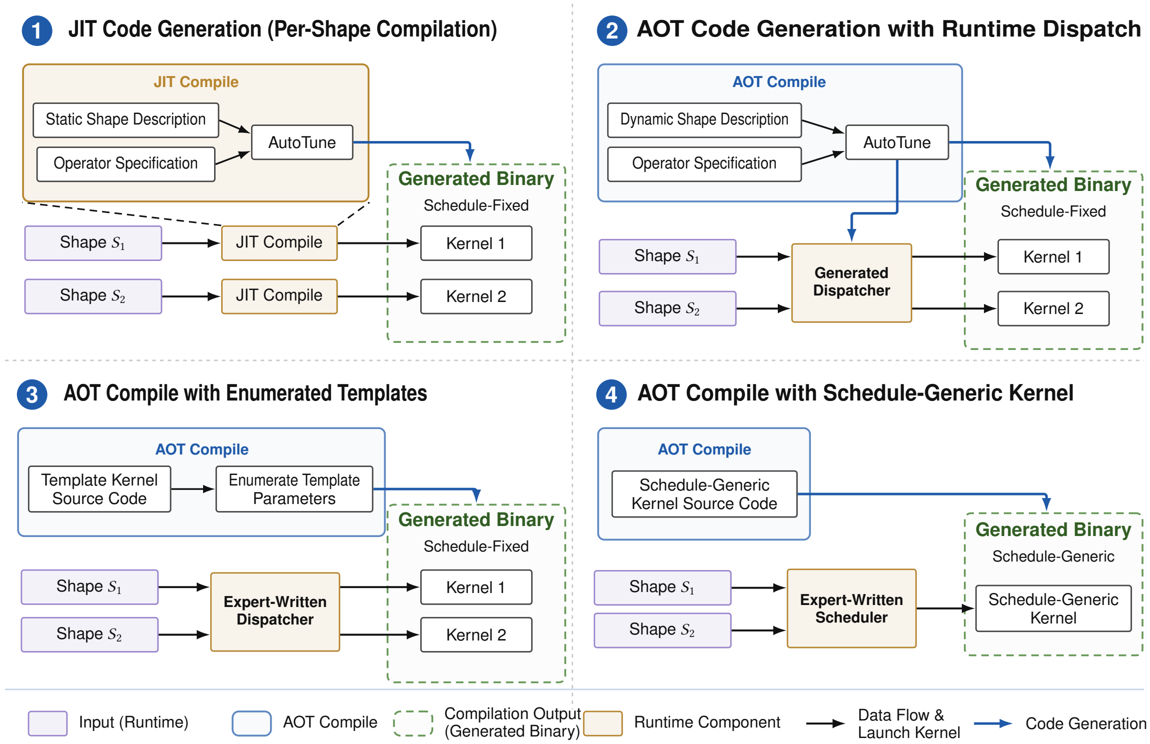


Fig. 1. Four ways to compile and execute dynamic-shape operators.

shape range ahead of time. It generates a set of optimized, schedule-fixed kernels together with a dispatcher that maps each runtime shape in the range to an appropriate kernel [17, 30, 43, 46].

~~On GPU platforms, which are highly sensitive to compiler optimizations like loop unrolling, developers usually resort to manual specialization using C++ templates~~ [7, 8, 40] ~~(depicted as ❶). Conversely, on architectures like Ascend~~ [19] ~~that provide diverse programmable units, the schedule parameters are substantially more numerous. Since manually specializing for such a vast search space is intractable, the vendor libraries (e.g., CANN~~ [4]) ~~on such platforms primarily invoke generic kernels (depicted as ❷), resulting in performance loss due to weakened compiler optimizations.~~

Expert-written operator libraries implement the other two approaches. ③ A schedule-fixed expert-written operator library encodes schedule parameters as C++ template parameters and enumerates their candidate values during ahead-of-time (AOT) compilation [7, 8, 40]. This process produces a set of schedule-fixed kernels, from which an expert-written dispatcher selects a template instance for each runtime shape. ④ A schedule-generic expert-written operator library instead AOT-compiles a schedule-generic kernel. At runtime, an expert-written host-side scheduler applies heuristic rules to the input shape, computes the schedule parameters, and passes them to the device kernel as runtime arguments.

~~Given these limitations, we focus on hand-written operator libraries and aim to address the shortcomings of both manual specialization and generic kernel approaches: achieving automatic specialization for generic libraries to recover performance loss, and pruning manually specialized libraries to mitigate combinatorial explosion and code bloat.~~

AI compilers such as Triton and TileLang complement expert-written operator libraries but operate on a different compilation path [33, 34]. These code-generation workflows integrate specialization into the multi-level lowering of a high-level operator program. During lowering, the shape and schedule values selected for specialization are materialized and propagated as constants in the intermediate representations. This process generates a new specialized kernel implementation but does not directly specialize the existing C++/LLVM implementation of an expert-written operator library.

~~To accommodate such dynamics, operator libraries typically adopt a generic implementation strategy: they calculate schedule parameters (configuration values such as tiling sizes and loop bounds) at runtime on the host side and pass them to the kernel function as variable arguments. However, this generic design comes at a cost. From the perspective of the underlying device native compiler, these critical configuration values appear as unknown variables rather than compile-time constants. This ambiguity effectively weakens aggressive compiler optimizations that rely on static information, such as constant folding, loop unrolling, and instruction schedule. Consequently, this often leads to suboptimal performance. In this context, specializing kernels by replacing input schedule parameters with constants at compile time is generally beneficial, as it can enhance subsequent compiler optimizations. As the complexity of operators~~ [8, 40] ~~continues to increase and programmable units~~ [14, 19] ~~within hardware become increasingly diverse, the number of schedule parameters of kernels explodes, further necessitating kernel specialization. While manual specialization resolves schedule parameters at compile time, exhaustively enumerating parameter combinations to accommodate input dynamics leads to combinatorial explosion and severe code bloat.~~

The two expert-written library designs expose a common <u>specialization trade-off</u>. Schedule-fixed kernels expose schedule parameters as compile-time constants, enabling optimizations such as constant folding, loop unrolling, and instruction scheduling; however, enumerating combinations of schedule parameters causes combinatorial code-size growth. Schedule-generic kernels use a compact binary to cover a large shape space, but representing schedule parameters as runtime variables limits the same compiler optimizations and can reduce performance. This trade-off becomes more pronounced as operator implementations grow more complex [8, 40] and accelerator architectures expose increasingly diverse programmable units [14, 19]. Both trends increase the number of schedule parameters and the size of their configuration space, further exacerbating the trade-off.

~~To address these limitations, we observe that deploying a specific model exhibits partial dynamics: while input dimensions (e.g., batch size, sequence length) vary at runtime, model-specific attributes (e.g., hidden size, head number) remain invariant throughout serving. We denote these compile-time constants as model-specific constants, which can be resolved by symbolic shape inference in current DL frameworks~~ [3, 17, 30, 46]~~. A key insight is that kernel schedule configurations are primarily determined by these model-specific constants. Our analysis (§2) shows that over 90% of schedule parameters become deterministic given model-specific constants, enabling static resolution of most dynamic schedule decisions.~~

Our key observation is that most schedule parameters are runtime invariants once a model is deployed. Most deployment scenarios exhibit <u>partial dynamics</u>: only a limited set of dimensions, such as batch size and sequence length, vary at runtime, while model-specific attributes and dimensions, such as hidden size, head count, and head dimension, remain fixed. We refer to these fixed facts as model-specific constants. Current DL frameworks can recover model-specific constants through graph capture and symbolic shape inference [3, 17, 30, 46]. Mechanistically, model-specific constants determine the core tiling and pipeline structure constrained by on-chip resources and compute units, whereas dynamic dimensions primarily affect the number of executions of these tiles, their parallel partitioning, and tail handling. Consistent with this mechanism, our analysis in §2 finds that, across the studied operators, about 90% of schedule parameters are runtime invariants determined by model-specific constants and do not depend on the concrete runtime values of dynamic dimensions.

~~To address the above challenges, we introduce TilingInfer, the first system designed to enable precise constant propagation across language boundaries for kernel specialization. We construct an automated cross-language and whole-program analysis framework for automatic specialization of generic library kernels and code debloating for manually specialized libraries. By introducing mechanisms specific to DL systems, TilingInfer extends traditional static analysis techniques and consists of four main components: (1) Cross-Language Context Reconstruction (§3.2) defines semantic mapping rules from Graph IR parameter types to the typed records in C++ memory, automatically generating partially-evaluated calling contexts without analyzing complex FFI glue code; (2) Early-Exit Edge Identification (§3.3) tracks the value flow of return values and determines whether a path leads to error status codes based on guard conditions, enabling precise pruning of infeasible edges at control-flow merge points; (3) Inter-Procedural Constant Propagation (§3.4) performs field-sensitive analysis to propagate constants through complex host-side programs; (4) Kernel Specialization and Library Pruning (§3.5) performs LLVM IR-level transformations to specialize generic kernels and eliminate redundant template instantiations.~~

This observation suggests a distinct application strategy for addressing the specialization trade-off: using model-specific constants to specialize existing operator libraries. Inspired by this insight, we present **TilingInfer**, a static-analysis framework for LLVM-based operator libraries. It uses LLVM IR as a common representation of C++ operator libraries. TilingInfer first uses graph-level model constants to **infer** invariant **tiling** data (i.e., schedule parameters) for both schedule-fixed and schedule-generic implementations. For each Graph IR node (i.e., an operator invocation), TilingInfer extracts its arguments, shapes, and attributes to construct the entry state of the corresponding host-side LLVM IR function. This construction covers both concrete and symbolic dimensions. TilingInfer then propagates this constructed state information through the host-side LLVM IR programs to device-kernel launch sites. Finally, for a schedule-generic kernel, TilingInfer employs the inferred launch-site constant schedule parameters to specialize the kernel. For the schedule-fixed implementation, TilingInfer identifies template instances that cannot be reached and removes these instances.

~~Based on this insight, we propose leveraging the model-specific constants as contextual information to perform whole-program constant propagation. By propagating these model-specific constants through the operator library down to the kernel call sites, we can statically specialize generic kernels or prune redundant instances in manually specialized libraries. However, implementing this strategy faces two critical challenges regarding the precision of static analysis: the cross-language semantic gap and early-exit paths from dynamic shape checks. First, the operator library is typically integrated into the framework via Python/C++ Foreign Function Interface (FFI), which creates a semantic gap between high-level Python objects and low-level C++ memory states. Standard static analysis cannot bridge this gap, as existing tools treat FFI glue code as black boxes, failing to model the deep heap layouts and internal indirections of domain-specific objects (e.g., Tensor's nested size arrays) that are essential for reconstructing precise C++ calling contexts. Second, the host-side programs of operators involve many shape checkers for validiting input shape, introducing numerous early-exit paths that return error status codes when shape constraints are violated. Standard static analysis conservatively merges data flows from these early-exit paths with the normal path, thereby corrupting precise constant information and causing significant precision loss at control-flow merge sites.~~

Realizing TilingInfer faces two key challenges. **First**, reconstructing the calling context requires bridging **a cross-language semantic gap** by mapping graph-level constants and symbolic shape information into the field-sensitive entry state of the corresponding C++ operator. Graph IR records constants and symbolic dimensions as typed shapes and attributes, whereas the separately compiled C++/LLVM code exposes nested object layouts and pointer/field accesses. Bridging this gap through end-to-end cross-language analysis would require traversing framework dispatch, type conversion, object construction, and other Python/FFI machinery, making the analysis costly and framework-specific. TilingInfer instead uses type-specific context-update rules to precisely construct the required C++ entry state directly from Graph IR, avoiding analysis of Python/C++ FFI glue code. **Second**, operator host code contains many **shape checkers that create early-exit paths**. These paths handle only invalid inputs: they report errors and bypass the normal computation of schedule parameters. As a result, merging states from early-exit and normal paths can pollute the normal-path state and reduce the precision of constant propagation. Meanwhile, analyzing every possible path separately would be too expensive for a large operator library. TilingInfer therefore uses a Conditional Return-Value Flow Graph (CRVFG) to identify early-exit edges from the conditional flow of status return values before propagation. It excludes states propagated along these edges from normal-path merges. This preserves constants without analyzing every path separately.

~~In summary, the main contributions of this paper are as follows:~~

(1) ~~We propose TilingInfer, a novel static analysis framework that enables precise constant propagation across the DL framework's language boundary (Python/C++) into the underlying operator libraries.~~
(2) ~~We introduce a cross-language context reconstruction method based on semantic modeling of core DL data types, and a conditional return-value flow analysis for identifying early-exit edges.~~
(3) ~~We implement TilingInfer based on LLVM [18] and apply it to PyTorch [3], with extensibility to other Python-based frameworks and hardware backends.~~
(4) ~~Evaluation demonstrates TilingInfer's effectiveness: for generic operators on Ascend, it achieves an average speedup of 1.06× (up to 1.17×) and 3% end-to-end inference speedup; for manually specialized libraries on GPU, it removes over 96% of redundant kernels.~~

This paper makes four contributions:

(1) We devise a mechanism that propagates graph-level shape information into the C++ compiler pipeline without analyzing FFI glue code. We formalize context-update rules that map Graph IR parameter types to the memory states of their underlying C++ implementations, allowing the mechanism to cover different PyTorch backends.
(2) We propose CRVFG-based early-exit path identification for precise constant-propagation analysis of operator host code while avoiding expensive path-sensitive analysis.
(3) We implement TilingInfer, a prototype system for specializing expert-written operator libraries on GPU and NPU platforms. It applies automatic partial evaluation to schedule-generic kernels to reduce scalar overhead and prunes unreachable template instances from schedule-fixed binaries at compile time to reduce binary size.
(4) On an Ascend NPU, TilingInfer achieves a 1.06× geometric-mean device-kernel speedup across 14 CANN operators and a 1.05× geometric-mean end-to-end speedup across six representative inference scenarios; it provides larger benefits in the performance-critical LLM decode phase, averaging a 1.10× end-to-end speedup across the two decode scenarios. On an NVIDIA GPU, it reduces a 542 MB FlashInfer archive by 88.8–95.5% across 17 model variants.

2 Background and Motivation

2.1 Graph IR and External Operator Libraries

~~In this section, we use a simplified Flash Attention (FA) operator as an example. Given query, key, and value tensors~~ $\cancel{Q \in \mathbb{R}^{B\times h_q\times S_q\times d_k}}$, $\cancel{K, V \in \mathbb{R}^{B\times h_{kv}\times S_{kv}\times d_k}}$, ~~and an optional mask~~ $\cancel{M}$~~, the attention operation is defined as:~~

A Graph Intermediate Representation (Graph IR) is a typed, directed dataflow graph: nodes encode inputs, attributes, operator calls, and outputs; edges carry tensors or scalars; and metadata records types and shapes. Framework IRs, such as PyTorch FX and TensorFlow graphs, preserve calls and arguments, while AI compiler IRs, such as TVM Relay/Relax, XLA HLO/StableHLO, and Inductor IR, normalize operations for optimization and lowering [3, 17, 24, 27, 31]. We focus on framework IR operator nodes because they define calls to external expert-written operator libraries.

$$\cancel{\mathrm{Attention}(Q,K,V,M) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right)V}$$

~~where B is the batch size, h_q is the number of query heads, h_{kv} is the number of key-value heads, S_q and S_{kv} are sequence lengths, and d_k is the head dimension. The operator imposes strict shape constraints: for example, batch sizes must match across Q, K, and V, the number of query heads must be divisible by the number of key-value heads ($h_q \mod h_{kv} = 0$), and sequence lengths of K and V must be equal. Typical configurations include $h_q \in \{8, 16, 32, 40, \ldots\}$, $d_k \in \{64, 128, 256\}$, various mask types (none, causal, sliding window), attention variants (MHA, GQA, MLA), and position encodings (RoPE, ALiBi).~~

With `torch.compile`, Dynamo extracts PyTorch operations from Python bytecode into an FX Graph [3, 26]. An FX operator node has $N = (\texttt{op}, \texttt{target}, \texttt{args}, \texttt{kwargs}, \texttt{meta})$: `op`/`target` identify the call, `args`/`kwargs` hold actual arguments, and `meta` stores type and shape information. We use the following simplified Flash-Attention computation:

$$\mathrm{FA}(Q,K,V,M) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right)V, \tag{1}$$

where $Q \in \mathbb{R}^{B\times h_q \times S_q \times d_k}$, $K, V \in \mathbb{R}^{B\times h_{kv} \times S_{kv} \times d_k}$, and M is an optional mask [8]. In Equation 1, B, S_q, and S_{kv} may vary at runtime, whereas h_q, h_{kv}, d_k, and attention attributes are model-fixed. A corresponding FX node has `op` = `call_function`, `target` = `aten.scaled_dot_product_attention`, `args` = `(q,k,v)`, and `kwargs` = {`attn_mask=None`, `dropout_p=0.0`, `is_causal=True`}. For the concrete model instance used in our running example, the input FakeTensors q, k, and v have shapes $(s_B, 40, s_q, 128)$, $(s_B, 8, s_{kv}, 128)$, and $(s_B, 8, s_{kv}, 128)$, respectively, while the output FakeTensor in `meta["val"]` has shape $(s_B, 40, s_q, 128)$. Here, s_B, s_q, and s_{kv} are `SymInt` symbols tracked by `ShapeEnv`, whereas 40, 8, and 128 are concrete integers.

~~These parameters are fixed per model but vary across models, making them ideal for compile-time specialization.~~

Example 2.1 (Scaled Dot-Product Attention Operator Signature). The simplified C++ operator signature is:

```
scaled_dot_product_attention(query: Tensor, key: Tensor, value: Tensor, attn_mask:
    Optional[Tensor], dropout_p: float, is_causal: bool) -> Tensor
```

It binds q, k, and v to `query`, `key`, and `value`, and the keyword values to `attn_mask`, `dropout_p`, and `is_causal`; other optional parameters are omitted. Inside the C++ host implementation, every dimension is read uniformly through calls such as `q->size(i)`. Such loads do not preserve whether the FX dimension was a concrete integer or a `SymInt`; the operator-library compiler therefore treats every loaded size as a runtime variable. This loss of graph-level shape information motivates its propagation into the C++ compilation flow.

2.2 Motivation of Automatic ~~Kernel~~Model-Specific Specialization

~~This section elucidates the opportunity for kernel specialization by tracing the flow of constant values across the software stack. It further demonstrates the necessity of this approach by analyzing the bottlenecks inherent in generic kernels on Ascend~~ [19]~~, a widely employed AI accelerator.~~

In existing expert-written operator-library compilation pipelines, treating constant dimensions and operator attributes as variables creates two forms of waste: scalar and control overhead in schedule-generic kernels and unreachable template instances in schedule-fixed binaries. This subsection examines the opportunity for Graph-to-C++ constant propagation, the limitations of current compilation pipelines, and the key observation that most scheduling parameters are runtime invariants.

2.2.1 ~~Cross-Layer Constant Value Flow~~*Graph-to-C++ Constant Propagation.* ~~The partial dynamics characteristic of DL models implies that while input tensors vary, there are many model-specific constants.~~ Figure 2 ~~illustrates how these constants propagate through the software stack. When a Python-level operator (e.g., torch.fa) is invoked, arguments flow through the FFI layer to the host C++ library. While some parameters are runtime-dependent (e.g., batch size %B and sequence length of query %S), others are derived from the fixed model configuration (e.g., number of query heads $h = 40$). As shown in the figure, within the host C++ runtime, the software pipeline depth stages is computed as stages = (h > 32) ? 2 : 1. Crucially, since stages depends solely on h, a constant derived from the model configuration, its value can be statically determined at compile time through constant propagation. By propagating $h = 40$ from the Python front-end to the host C++ compiler, the expression (h > 32) ? 2 : 1 can be evaluated to yield stages = 2, enabling the compiler to specialize the generic kernel kernel_fa into a concrete variant kernel_fa2, where the suffix denotes the specialized stages value.~~

~~This observation suggests an opportunity: by statically tracking model-specific constants from the Python front-end down to the kernel launch, we can identify runtime invariants that are currently treated as variables. For generic kernels, this enables aggressive specialization of the device code. Furthermore, for manually specialized libraries, this allows us to identify the specific kernel instantiations not matching the derived compile-time constants. This facilitates the pruning of unreachable code paths for a specific model, mitigating binary bloat.~~

DL models are partially dynamic: batch size and sequence lengths may vary at runtime, while dimensions and attributes such as head count are fixed by the model configuration. When a Python operator is invoked, its tensors and attributes cross the FFI boundary into the C++ host library, but their model-fixed status is not available to the separately compiled library. Figure 2 illustrates a fixed head count `h=40` flowing into the host expression `stages = (h > 32) ? 2 : 1`. Propagating `h=40` across the language boundary allows host-side constant propagation to infer the invariant schedule parameter `stages=2`. Making such schedule invariants available at compile time creates an opportunity to specialize schedule-generic kernels and discard incompatible pre-instantiated variants.

2.2.2 ~~Disadvantages of Generic Kernels and Manually Specialized Libraries~~*Limitations of the Two Expert-Written Operator-Library Designs.*

Bottleneck Analysis ~~for Generic Kernels~~*of Schedule-Generic Implementations.* ~~Some operator libraries implement their kernels in the form of generic kernels, such as the built-in operator library of Ascend CANN. These kernels are not specialized mainly due to their complex schedule strategies and the great number of schedule parameters. In this design, almost all schedule parameters are passed as runtime arguments from the host. Since the compiler treats these parameters as variables, it cannot perform aggressive optimizations like constant folding. To understand the impact of this design, we examine the execution flow on Ascend, a DL accelerator based on SIMD execution model, as shown in~~ Figure 3~~.~~

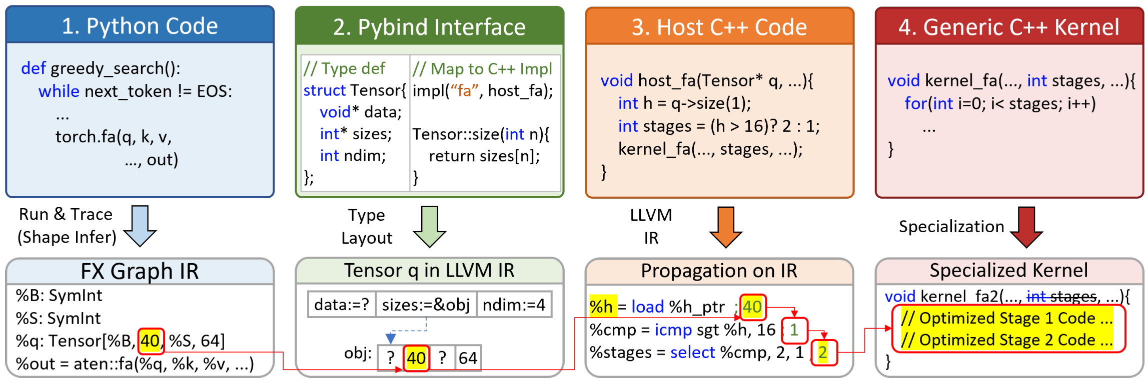


Fig. 2. A simplified Graph-to-C++ constant-propagation opportunity for Flash Attention. A model-fixed graph dimension (e.g., `q->size(1) = 40`) flows into C++ host code and determines the launch-site schedule parameter `stages = 2`, enabling specialization of `kernel_fa` into `kernel_fa2`.

Schedule-generic kernels support complex scheduling strategies by receiving numerous schedule parameters from the host at runtime. Because the compiler cannot treat these parameters as constants, optimizations such as constant folding and loop simplification remain limited. Figure 3 illustrates the resulting overhead on Ascend.

~~Ascend is a DL accelerator based on SIMD execution model, featuring diverse compute and storage units for handling different types of tensor computations and data movement operations. Specifically, the Cube and Vector units execute the primary tensor operations (e.g., MatMul on Cube and Softmax on Vector), while the architecture provides sophisticated programmable memory hierarchies. The kernel must explicitly manage the chunk size and data layout at every level, as well as the data movement and layout transformations between these levels. Besides, the Scalar unit acts as the central control unit, responsible for scalar data processing, instruction dispatch, and handling control flows (such data paths are indicated by red dashed lines; for simplicity, only some are shown). For example, the kernel moves a tile of the tensor Q_{GM} from the global memory (GM) to the L1 memory unit at a time, denoted as Q_{L1}. During this data movement, the Scalar unit is responsible for calculating the starting memory address of tile Q_{GM}, implementing loop iterations, and loading or storing necessary scalar data (including schedule parameters and partial scalar intermediate results) from the global memory or the Unified Buffer (UB).~~

~~In generic kernels, the Scalar unit may be over-occupied. Since the Scalar unit typically possesses much lower computing power than a general-purpose host CPU, excessive scalar calculations related to schedule parameters can overload the unit. Worse yet, excessive scalar intermediates may cause register spill, as there are only 32 general-purpose registers. As a result, the powerful Cube and Vector units will stall to await instruction dispatch or scalar operands. Furthermore, generic kernels often involve complex control flows (e.g., branching to handle different operator variants), which exerts heavy pressure on the instruction cache (I-Cache). Frequent instruction jumps in such complex logic inevitably lead to I-Cache misses and pipeline flushes. Finally, when loop bounds remain unknown at compile time, the compiler struggles to perform loop unrolling. Consequently, instruction scheduling is confined to the scope of a single iteration, preventing the effective overlapping of memory access and computation.~~

~~We further study this impact by profiling key operators from the Qwen2.5-1.5B model on the Ascend NPU. Figure 4 presents the Scalar unit occupancy (the proportion of active cycles during kernel execution) and I-Cache miss rates. The data reveals that the Scalar unit is active for 78% to 91% of total cycles, while I-Cache miss rates range from 3.1% to 16.4%.~~

Ascend combines Cube and Vector compute units with a programmable memory hierarchy. Kernels explicitly manage tile sizes, data layouts, and data movement across memory levels, while the Scalar unit computes addresses and loop bounds, handles control flow, and dispatches instructions. For example, when moving a tile from Q_{GM} in global memory (GM) to Q_{L1} in L1 memory, the Scalar unit determines its starting address, advances the loop, and accesses schedule parameters and scalar intermediates in GM or the Unified Buffer (UB). The representative profiles in Figure 4 show high Scalar ratios and nonzero I-Cache miss rates for several of these kernels. These measurements motivate device-side specialization to reduce the scalar workload and improve instruction locality.

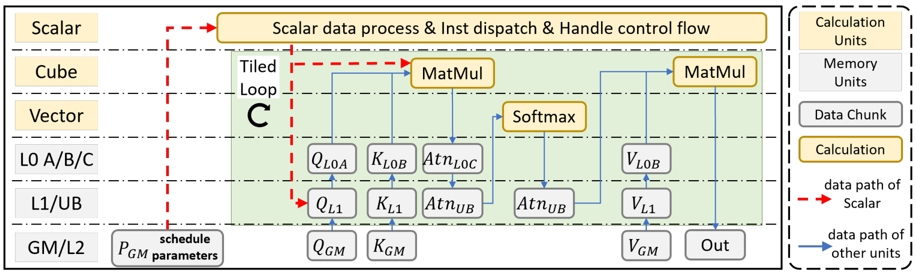


Fig. 3. Execution flow of Flash Attention kernel `kernel_fa` on the Ascend architecture. The **Scalar** unit acts as the central scheduler. It continuously performs scalar calculations and dispatches instructions based on runtime parameters. The heavy scalar workload required by generic kernels may lead to a scalar bottleneck.

Code Bloat in ~~Manually Specialized Libraries~~Schedule-Fixed Implementations. High-performance GPU libraries ~~like~~ such as FlashInfer [40] rely on C++ template metaprogramming to enable aggressive compiler optimizations. To avoid runtime JIT overhead, ~~these~~ such libraries pre-instantiate kernels for the Cartesian product of all template parameters. As described in Equation 1, FA's configuration space includes head dimensions ($d_k \in \{64, 128, 256\}$), mask types, GQA sizes, and position encodings. Combined with ~~tile size~~tile-size configurations, this results in over ~~1000~~1,000 binary instantiations, yet a specific model ~~like Qwen2-1.5B utilizes~~uses only ~10 kernels. This pre-instantiation strategy, while manageable on GPUs, is ill-suited for Ascend due to its more complex schedule parameters, forcing ~~Ascend libraries~~ their Ascend counterparts to rely on ~~suboptimal generic kernels~~schedule-generic kernels with limited compile-time optimization.

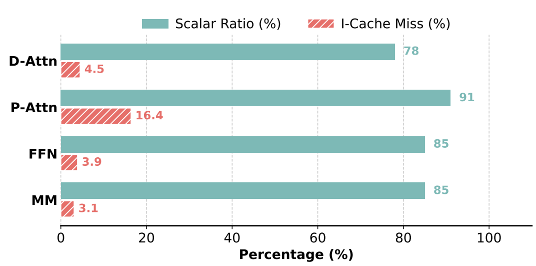


Fig. 4. Performance profiles of schedule-generic kernels, where lower values are better for both metrics: Decoding Attention (**D-Attn**), Prefill Attention (**P-Attn**), Feed Forward Networks (**FFN**), and MatMul (**MM**).

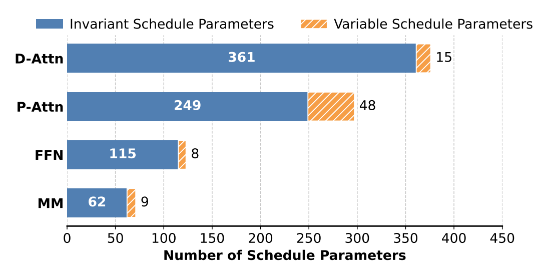


Fig. 5. Number of schedule parameters of representative kernels. The vast majority are runtime invariants (Invariable Params), validating the potential for specialization.

2.2.3 Why Is Automatic ~~Kernel~~ Model-Specific Specialization Beneficial? We further analyze the schedule parameters of key operators from the **Qwen2.5-1.5B** model, classifying them into *invariable parameters* (constants derived from model configuration) and *variable parameters* (runtime-dependent values such as batch size and sequence length). Figure 5 reveals that the overwhelming majority of kernel parameters are invariable. Propagating model-specific constants to resolve schedule parameters at compile time yields two benefits: (1) for ~~generic kernels~~schedule-generic implementations, it enables model-specific kernel specialization with aggressive constant folding, reducing scalar overhead and I-Cache misses; (2) for ~~manually specialized libraries~~schedule-fixed implementations, it makes kernel dispatch paths deterministic, enabling precise pruning of unused binaries.

In addition, the sensitivity study in §4.4 analyzes which schedule parameters have the greatest effect on performance. It shows that a small subset of parameters on the kernel's hot path accounts for most of the performance gain.

2.3 Challenges and Design Insights

~~Achieving automatic kernel specialization requires analyzing the value flow from Python down to C++, which presents two major challenges that cannot be properly addressed by existing static analysis.~~

Achieving automatic model-specific specialization requires transferring graph-level facts into C++ compilation and preserving them through host-side control flow. These steps present two challenges not directly addressed by existing static analyses.

2.3.1 Challenge 1: ~~The cross-language semantic gap~~Reconstructing Cross-Language Calling Contexts. ~~Achieving automatic kernel specialization requires reconstructing the calling context for C++ operator functions, i.e., determining the tensor shapes and operator attribute values. As shown in~~ Figure 2~~, these values are instantiated on the Python side and passed through the FFI layer to C++. However, reconstructing this cross-language value flow faces significant challenges.~~

The first challenge is the Python/C++ semantic gap: Graph IR constants and symbolic dimensions must be mapped to the memory states of their underlying C++ objects before LLVM can use them as compile-time facts. This mapping is not a direct binding between the arguments of a Graph IR node and C++ parameters. Graph IR records typed values and shapes, whereas composite C++ parameters such as `Tensor`, `List`, and `Optional` use nested object layouts; after lowering, an access such as `query->size(1)` becomes pointer arithmetic and `load` instructions. The analyzer must therefore reconstruct the corresponding base objects, pointers, field offsets, and field values, preserving concrete dimensions as constants and symbolic dimensions as unknown.

~~Existing cross-language analysis methods~~ [15, 20] ~~typically employ abstract interpretation at the FFI API level, modeling each cross-language call as state transitions in an abstract domain. While general-purpose, this approach is ill-suited for DL systems for two reasons. First, the FFI layer contains substantial glue code for type conversions, dynamic dispatch, and reference counting, operations that are essential for language interoperability but irrelevant to the constant values we aim to extract. Analyzing this glue code is both computationally expensive and unnecessary for our goal. Second, the Python-side Graph Executor, which orchestrates operator invocations based on Graph IR, comprises complex logic spanning thousands of lines of code. Performing full abstract interpretation over this codebase is prohibitively expensive. These factors render traditional approaches impractical for reconstructing operator calling contexts in DL systems.~~

Existing cross-language analyses broadly follow two approaches: multilanguage abstract interpretation jointly analyzes programs on both sides of an FFI boundary [20], while specification-based methods summarize native behavior under a cross-language caller context [15]. The former would have to traverse the Python Graph Executor and FFI glue for dispatch, type conversion, object construction, and reference counting, which is expensive and largely irrelevant to graph-level constants. The latter assumes or derives a cross-language caller context but does not define how Graph IR metadata should materialize the field-sensitive C++ entry state required by LLVM.

~~Insight: Semantic modeling of core data types in DL systems. Instead of analyzing FFI glue code and Python source code, we propose a cross-language semantic modeling approach grounded in two observations: (1) Finite and Stable Core Types: Data types used for cross-language interaction in DL systems are highly convergent and stable (Tensor, Array, etc.), enabling comprehensive and precise semantic modeling. (2) Extract Constant Information from Graph IR: DL systems compile models and represent them using Graph IR (e.g., ATen IR), which contains precise information regarding operator input parameters, enabling direct extraction of semantic values without analyzing Python source code. Specifically, model-specific constants are captured as exact values from tracing, while dynamic dimensions are represented as symbolic variables. Building on these insights, we directly map computation graph node information to the abstract states of C++ entry functions, achieving high-precision reconstruction of C++ contexts while avoiding the overhead of analyzing complex FFI glue code.~~

Insight: Type-directed calling-context construction. DL operator interfaces use a limited set of parameter types, and their C++ interface layouts are stable within a backend. We can therefore define a context-update rule for each type and use Graph IR shapes and attributes to construct an accurate C++ operator calling context directly. This approach avoids analyzing FFI glue code; Table 2 formalizes the context-update rules used by TilingInfer.

```
const status SUCCESS=0, ERROR=-1;
status check(int s1, int s2) {
  if(s1 != s2) return ERROR;
  return SUCCESS;
}
status set_t(int sk, int sv, int* t) {
  int e = check(sk, sv);
  if(e != SUCCESS) {
    print("error"); return e;
  }
  *t=32; // Tiling
  return SUCCESS;
}
```

Code 1. Early-exit path example.

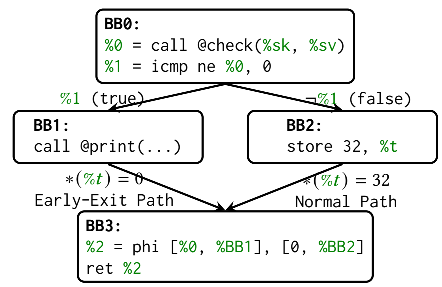


Fig. 6. Control Flow Graph (CFG) of set_t's LLVM IR. Each node is a basic block containing LLVM IR instructions. Edges are labeled with branch conditions.

2.3.2 Challenge 2: Shape Checkers Leading to Early-Exit Paths.

~~Operator libraries enforce shape constraints through runtime shape checkers to ensure memory safety and computational correctness. As defined in~~ Equation 1~~, the Flash Attention operator imposes one constraint that the sequence lengths of K and V must be equal ($S_k = S_v$). Violations of these constraints trigger early exits with error codes.~~

The second challenge is preserving constant-propagation precision across shape-checker-induced early-exit paths without expensive path-sensitive analysis. Operator libraries enforce constraints for memory safety and correctness; for example, Flash Attention requires equal key and value sequence lengths ($S_k = S_v$). Because these dimensions may remain symbolic, the compiler cannot decide whether a check succeeds. An early-exit path then bypasses assignments performed on the normal path, and merging the two states can obscure constants.

Code 1 ~~illustrates this pattern with a simplified example. The function check verifies whether two sequence lengths (denoted as s1 and s2) match, returning SUCCESS (0) if they are equal or ERROR (-1) otherwise. The function set_t~~

~~uses this checker to validate the sequence lengths before computing the tiling parameter t=32. Since these dimensions are dynamic at compile time, the compiler cannot determine whether the check will succeed or fail.~~

Figure 6 ~~shows the corresponding CFG. The entry block BB0 calls @check and branches based on the result: the true branch leads to BB1 (error path), while the false branch leads to BB2 (normal path) where t=32 is defined. Both paths converge at BB3. At this merging site, standard path-insensitive analysis cannot determine whether the definition of t originates from BB2 or is undefined from BB1, resulting in a polluted abstract state~~ ~~$\{t \in \{0, 32\}\}$~~ ~~that prevents constant propagation.~~

In Code 1, `check` returns `SUCCESS` or `ERROR`, and `set_t` computes `t=32` only after the check succeeds. The CFG in Figure 6 branches at **BB0**: **BB1** returns through the early-exit path without defining `t`, whereas **BB2** defines it before both paths reach **BB3**. If `t` initially holds 0, path-insensitive merging yields $\{0, 32\}$ rather than the normal-path constant 32. Enumerating all paths would avoid this pollution, but nested shape checkers make full path-sensitive analysis impractical.

~~Insight: Conditional Return-Value Flow. The key to identifying early-exit paths lies in analyzing the relationship between path guards and the semantic meaning of return values. Since shape checkers assign different constant values to the return variable on different execution paths (e.g., SUCCESS on the normal path and ERROR on the error path), we can determine whether a path is an early-exit by examining whether the path guard implies an error return value. Specifically, we maintain path-sensitive guards and propagate them through the control flow. If a path guard implies that the return value equals an error code, the path is identified as an early-exit path; otherwise, it is conservatively treated as a normal path. During constant propagation, value states from confirmed early-exit paths are prevented from propagating to merge points, thereby avoiding state pollution. In~~ Figure ~~6, the branch condition at BB0 is %1 = (%0 ≠ 0), where %0 is the return value of @check. When the guard %1 = *true* holds, we have %0 ≠ 0, which implies %0 ≠ *SUCCESS*; hence the return value represents an error, and the path *BB0 → BB1 → BB3* guarded by %1 is an early-exit path. Conversely, when the guard ¬%1 holds, we have %0 = 0 = *SUCCESS*; hence the return value represents success, and the path *BB0 → BB2 → BB3* guarded by ¬%1 is a normal path.~~

Insight: Conditional Return-Value Flow. CRVFG-based normal-path reasoning traces only the conditional flow of status return values and compares them with configured success and error sets. In the example, the guard $\%1 = (\%0 \neq 0)$ being true implies that `check`'s return value is not `SUCCESS`, so **BB0→BB1** is an early-exit edge; its false branch is the normal edge. Confirmed early-exit-edge states are excluded from normal-path merges, while unclassified edges are retained conservatively.

3 System Design of TilingInfer

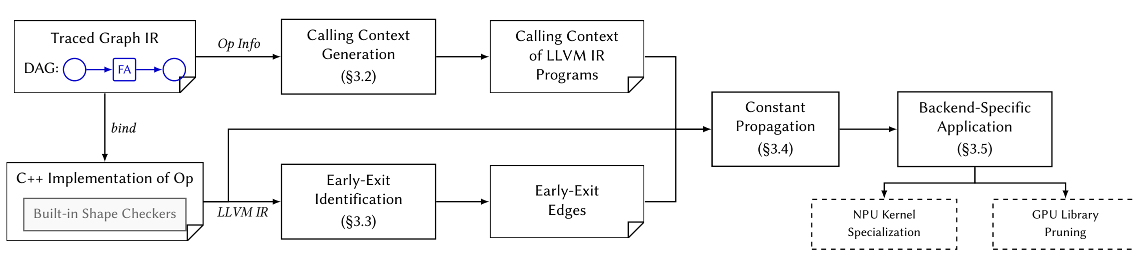


Fig. 7. The overall workflow of TilingInfer.

Figure 7 illustrates the workflow of TilingInfer. Here, an early-exit edge is the CFG edge that enters an early-exit path after a shape check fails. The framework takes the traced Graph IR containing operator invocation information with model-specific constants as input, and proceeds through four stages: **Context Generation** (§3.2), **Early-Exit Identification** (§3.3), **Constant Propagation** (§3.4), and **Kernel Specialization** (§3.5). We detail each stage in the following subsections.

Table 1. Abstract domains and definitions in TilingInfer.

Domain	Notation	Domain	Notation
Top-level Variable	$v \in \mathcal{V}$	Constant Value	$c \in \mathbb{Z} \cup \mathbb{R} \cup \mathcal{F}$
Base Object	$b \in \mathcal{B}$	Abstract Pointer	$p := \langle b, \hat{\delta} \rangle$
Function Object	$f \in \mathcal{F}$	Specialized Value	$sv := c \mid p \mid \top \mid \bot \in \mathcal{SV}$
Concrete Offset	$\delta \in \mathbb{Z}$	Environment	$\mathbb{E} := \mathcal{V} \mapsto \mathcal{SV}$
Abstract Offset	$\hat{\delta} \in \mathbb{Z} \cup \{\bot\}$	Store	$\mathbb{S} := \mathcal{O} \mapsto \mathcal{SV}$
Memory Location	$o := \langle b, \delta \rangle \in \mathcal{O}$	Calling Context	$\mathbb{C} := (\mathbb{E}, \mathbb{S})$

3.1 Definitions

To formally describe the analysis of TilingInfer, we first define the abstract domains and notations used in our framework, as summarized in Table 1.

Memory Model and Variables. We distinguish between SSA values and memory locations:

- *Top-level variables* ($v \in \mathcal{V}$) correspond to virtual registers in the LLVM IR.
- *Base objects* ($b \in \mathcal{B}$) represent the abstract base addresses of memory blocks created at allocation sites (e.g., `alloca` instructions, global variables, or heap allocations).
- *Memory locations* ($o \in \mathcal{O}$) represent specific fields within allocated objects. A location is defined as a pair $\langle b, \delta \rangle$, where $\delta \in \mathbb{Z}$ denotes a concrete byte offset relative to the base address b.

Abstract Values and Lattice. The analysis tracks the state of variables using *Specialized Values* ($sv \in \mathcal{SV}$). The domain $\mathcal{SV}$ forms a lattice comprising the following elements:

- *Top ($\top$) and Bottom ($\bot$):* $\top$ represents an *undefined* or *uninitialized* state (the lattice top, indicating no information), while $\bot$ represents a generic *variable* state where the value cannot be statically determined (the lattice bottom, indicating over-approximation).
- *Concrete Constants (c):* These represent precise, known values. This category includes integers ($\in \mathbb{Z}$), floating-point numbers ($\in \mathbb{R}$), and *Function Objects* ($f \in \mathcal{F}$), which represent references to functions targeted by direct or indirect calls.
- *Abstract Pointers (p):* Defined structurally as $\langle b, \hat{\delta} \rangle$. A pointer p represents an address computed by adding an *Abstract Offset* $\hat{\delta}$ to a base object b. Here, $\hat{\delta} \in \mathbb{Z} \cup \{\bot\}$ allows the analysis to track precise offsets when possible, or degrade to $\bot$ if the offset is variable (e.g., a runtime index).

Context and State. The program state and ~~inter-procedural~~ interprocedural context are captured by the following mappings:

- The *Environment* $\mathbb{E} : \mathcal{V} \mapsto \mathcal{SV}$ maps top-level virtual registers to their abstract values.
- The *Store* $\mathbb{S} : \mathcal{O} \mapsto \mathcal{SV}$ maps memory locations to the values stored therein. By mapping from pairs of $\langle b, \delta \rangle$, the store naturally supports field-sensitive tracking of structure fields and array elements.

- The *Calling Context* $\mathbb{C} := (\mathbb{E}, \mathbb{S})$ encapsulates the analysis state (environment and store) at the entry basic block of the current function. This context is determined by the caller and propagates constraints across ~~functions during inter-procedural~~ function boundaries during interprocedural analysis.

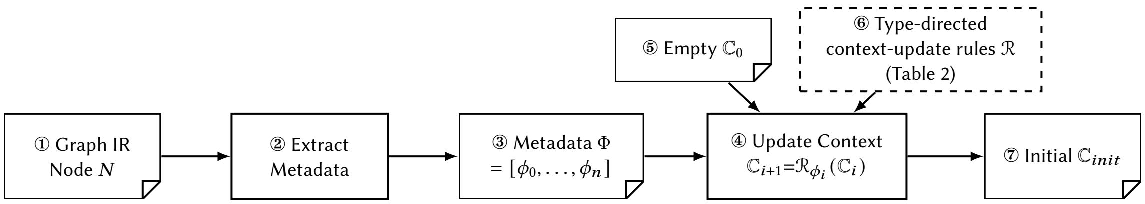


Fig. 8. Workflow of calling-context generation. ①–③: extraction parses the Graph IR node into typed metadata Φ. ④–⑦: the five type-directed context-update rules $\mathcal{R}$ in Table 2 map Φ to the abstract C++ calling context $\mathbb{C}_{init}$.

3.2 Calling Context Generation for C++

Figure 8 illustrates the overall workflow of context generation in TilingInfer. The process takes a Graph IR node N representing an operator invocation as input, and produces the initial calling context $\mathbb{C}_{init}$ for the corresponding C++ entry function. This process bridges the ~~semantic gap between the high-level Graph IR and the low-level~~ representation and compilation boundary between Graph IR metadata and the abstract C++ ~~execution state~~ entry state required by LLVM analysis through two phases: (1) **Extraction** (①–③), which parses the IR node N to ~~extract the parameter type information~~ generate structured metadata Φ; and (2) **Construction** (④–⑦), which applies ~~semantic rules~~ the context-update rules $\mathcal{R}$ to transform the empty context $\mathbb{C}_0$ into the initial calling context $\mathbb{C}_{init}$. Here, $\mathcal{R} = \{\mathcal{R}_{\text{SCALAR}}, \mathcal{R}_{\text{STRING}}, \mathcal{R}_{\text{TENSOR}}, \mathcal{R}_{\text{LIST}}, \mathcal{R}_{\text{OPTIONAL}}\}$ is the family of five type-directed context-update rules defined in Table 2. Each rule maps one typed metadata item to updates of the abstract environment and store, thereby materializing the C++ entry-state fields accessed by the target operator.

3.2.1 *Graph IR Node and ~~Parameter Type~~ Metadata Extraction.*

The Graph IR is a functional intermediate representation based on the ATen operator set, organized as a directed acyclic graph ~~(DAG) where~~ in which each node represents a logical ATen operator invocation. We formally define a Graph IR node as:

$$N = (\mathsf{op}, \mathit{In}, \mathit{Out}, \sigma) \tag{2}$$

where op is the ATen operator identifier (e.g., `aten.`~~fa~~`scaled_dot_product_attention`, `aten.linear`), $\mathit{In} = [x_1, \dots, x_m]$ is the ordered input list, $\mathit{Out} = [y_1, \dots, y_k]$ is the ordered output list, and σ is the shape environment. Each input x_i is either a tensor ~~(~~produced by another node ~~)~~ or an attribute~~(e.g., integer~~, ~~float, arrays)~~such as an integer, a floating-point value, or an array. The type of each input and output is directly represented by the corresponding metadata item ϕ_i in the context metadata Φ, which is derived from the operator signature. The shape environment σ maps each tensor value v to its type and shape information:

$$\sigma(v) = (\mathtt{dtype}(v), \mathtt{shape}(v)) \tag{3}$$

where $\mathtt{shape}(v) = [d_1, \dots, d_r]$ is a dimension list with each d_i being either a concrete integer or a symbolic integer (`SymInt`) representing a runtime-determined dimension. This design enables the Graph IR to capture dynamic shape information symbolically.

ATen operators have strictly defined signatures that specify the types of inputs and outputs. ~~To capture the type and value information of each parameter, we define the parameter type descriptor~~ Each parameter type is directly mapped to a metadata variant ϕ ~~as~~ (e.g., Tensor, Scalar, List, Optional) without introducing a separate type variable.

TilingInfer extracts ② the *context metadata* Φ ③ by aligning the Graph IR node's arguments with the operator's formal parameters. For each parameter in the signature, if an explicit argument is provided, TilingInfer captures its value or shape information as a metadata item ϕ_i according to the parameter's type; if the argument is omitted and a default value exists, the default is used.

The context metadata $\Phi = [\phi_0, \phi_1, \ldots, \phi_n]$ is a sequence where each ϕ_i is a tagged union ~~:~~capturing the type and value information:

$$\phi ::= \textsc{Scalar}(c) \mid \textsc{String}(s) \mid \textsc{Tensor}(\vec{d}) \mid \textsc{List}\langle \phi_{\underline{1}}{}' \rangle(\vec{e}) \mid \textsc{Optional}\langle \phi_{\underline{1}}{}' \rangle(v) \tag{4}$$

Each parameter type constructor corresponds to the semantic meaning in the C++ operator interface:

- $\textsc{Scalar}(c)$: Represents a scalar parameter of type Int or Float, where c is the concrete value.
- $\textsc{String}(s)$: Represents a string parameter, where s is the string content.
- $\textsc{Tensor}(\vec{d})$: Represents a tensor parameter, where $\vec{d} = [d_1, \ldots, d_k]$ is the dimension list with each d_i being either a concrete constant or a symbolic expression.
- $\cancel{List\langle\phi_1\rangle(\vec{e})}\textsc{List}\langle\phi'\rangle(\vec{e})$: Represents a list parameter of element type $\cancel{\phi_1}\phi'$, where $\vec{e}$ is the sequence of elements.
- $\cancel{Optional\langle\phi_1\rangle(v)}\textsc{Optional}\langle\phi'\rangle(v)$: Represents an optional parameter, where v is either a concrete value of type $\cancel{\phi_1}\ \phi'$ (present) or the special value None (absent).

~~organizes the parameter type descriptors of a Graph IR node into a parameter type sequence~~ $\cancel{\Phi = [\phi_0, \phi_1, \ldots, \phi_n]}$~~. The extraction process aligns the~~

Example 3.1. Consider the `aten.scaled_dot_product_attention` invocation and the simplified C++ operator signature introduced in example 2.1. The Graph IR node ~~'s arguments with the operator's formal parameters: for each parameter in the signature~~ is $N = (\texttt{aten.scaled\_dot\_product\_attention}, \mathit{In}, \mathit{Out}, \sigma)$, where $\mathit{In} = [\mathsf{q}, \mathsf{k}, \mathsf{v}, \mathsf{None}, 0.0, 1]$ follows the signature order, $\mathit{Out} = [\texttt{o}]$, and 1 encodes `is_causal=True`. The shape environment records:

$$\begin{aligned} \sigma(\texttt{q}) &= (\texttt{dtype(q)}, [s_B, 40, s_q, 128]), & \sigma(\texttt{k}) &= (\texttt{dtype(k)}, [s_B, 8, s_{kv}, 128]), \\ \sigma(\texttt{v}) &= (\texttt{dtype(v)}, [s_B, 8, s_{kv}, 128]), & \sigma(\texttt{o}) &= (\texttt{dtype(o)}, [s_B, 40, s_q, 128]). \end{aligned}$$

Thus, s_B, s_q, and s_{kv} remain symbolic, while 40, ~~if an explicit argument is provided, captures its value or shape information as a parameter type descriptor~~ $\cancel{\phi_I}$~~; if the argument is omitted and a default value exists, the default is used. This parameter type sequence serves as the input for the subsequent Context Construction phase.~~

~~Consider the aten.fa operator implementing the Flash Attention computation defined in : aten.fa(Tensor q, Tensor k, Tensor v, int? lens, int? mask_type) -> Tensor The Graph IR node~~ $\cancel{N = (aten.fa, In, Out, \sigma)}$ ~~where~~ $\cancel{In}$ ~~contains: query tensor~~ $\cancel{q}$ ~~of shape~~ $\cancel{[B, 40, S, 64]}$ ~~(with symbolic~~ $\cancel{B, S}$ ~~and concrete~~ $\cancel{h_q = 40, d_k = 64}$~~), key tensor~~ $\cancel{k}$ ~~and value tensor~~ $\cancel{v}$ ~~both of shape~~ $\cancel{[B, 40, S, 64]}$ ~~(concrete~~ $\cancel{h_{kv} = 40}$~~), an optional sequence lengths array, and an omitted optional mask type~~

~~(default None)~~8, and 128 are concrete. The extracted ~~parameter type sequence~~ metadata Φ is:

$$\Phi = [\textsc{Tensor}([\cancel{B}s_B, 40, \cancel{S}s_q, \cancel{64}128]), \textsc{Tensor}([\cancel{B}s_B, \cancel{40}8, \cancel{S}s_{kv}, \cancel{64}128]),$$
$$\textsc{Tensor}([\cancel{B}s_B, \cancel{40}8, \cancel{S}s_{kv}, \cancel{64}128]), \textsc{Optional}\langle \cancel{\text{ListScalar}}\textsc{Tensor}\rangle(\text{None}),$$
$$\cancel{\text{OptionalScalar}}\cancel{(\text{None})}\textsc{Scalar}(0.0), \textsc{Scalar}(1)].$$

~~Each parameter type descriptor ϕ_i directly captures the type and value of the corresponding i-th input. For tensor parameters, the dimension list captures both concrete constants (e.g., 40, 64) derived from the model configuration and symbolic expressions (e.g., B, S) representing runtime-determined dimensions.~~

3.2.2 *Context Construction from ~~Parameter Types~~Metadata.*

The goal of the construction phase is to synthesize the initial C++ calling context $\mathbb{C}_{init} = (\mathbb{E}, \mathbb{S})$ from the ~~parameter type sequence~~ metadata Φ. Starting with an empty context $\mathbb{C}_0$ ⑤, TilingInfer iterates ~~through each parameter type descriptor~~ over the metadata items ϕ_i and sequentially updates ④ the abstract state: for $\tau_i = \text{type}(\phi_i)$, we write $\mathcal{R}_{\phi_i}(\mathbb{C}) := \mathcal{R}_{\tau_i}(\mathbb{C}, \phi_i)$ for the instance of the matching type-directed context-update rule.

$$\mathbb{C}_{init} = \mathcal{R}_{\phi_n} \circ \cdots \circ \mathcal{R}_{\phi_1} \circ \mathcal{R}_{\phi_0}(\mathbb{C}_0) \tag{5}$$

~~For each parameter type descriptor ϕ_i, we apply a semantic modeling rule~~Thus, each $\mathcal{R}_{\phi_i}$ ~~to transform~~ in the equation is an application of exactly one of the five context-update rules in $\mathcal{R}$ shown in Table 2, transforming the high-level ~~parameter type~~ metadata into the low-level memory representation.

3.2.3 *~~Update~~ Context-Update Rules for ~~Parameter~~ Metadata Types.*

Table 2 presents the ~~update rulesfor each parameter type.~~ five context-update rules, one for each supported metadata type.

Table 2. The five type-directed context-update rules $\mathcal{R}$ for constructing the abstract C++ entry state. **Definitions:** alloc(T) creates an abstract base object with the validated fields accessed through type T. Process$(type, v, \ell)$ recursively populates the store at location ℓ with value v of type *type*. $\alpha(x)$ returns x if concrete, or $\perp$ if symbolic.

Context-Update Rule	Abstract Fields	Environment Update	Store Update
$\mathcal{R}_{\textsc{Scalar}}$ $\phi = \textsc{Scalar}(c)$	`int64_t c;` `double c;`	$v \mapsto \alpha(c)$	None

$\mathcal{R}_{\textsc{String}}$
$\phi = \textsc{String}(s)$
	`struct {`
	`size_t len;`
	`char* data;`
	`}`
		$\ell = \text{alloc(str\_view)}$
		$\ell_{buf} = \text{alloc(char}[\lvert s\rvert])$
		$v \mapsto \langle \ell, 0\rangle$
			$\langle \ell, \delta_{\text{len}}\rangle \mapsto \lvert s\rvert$
			$\langle \ell, \delta_{\text{data}}\rangle \mapsto \langle \ell_{buf}, 0\rangle$
			$\forall i.\ \langle \ell_{buf}, i\rangle \mapsto s[i]$

$\mathcal{R}_{\textsc{Tensor}}$
$\phi = \textsc{Tensor}(\vec{d})$
$\vec{d} = [d_1 \dots d_k]$
	`struct {`
	`int64_t* sizes;`
	`int64_t ndim;`
	`}`
		$\ell = \text{alloc(Tensor)}$
		$\ell_{sz} = \text{alloc(int64\_t}[k])$
		$v \mapsto \langle \ell, 0\rangle$
			$\langle \ell, \delta_{\text{ndim}}\rangle \mapsto k$
			$\langle \ell, \delta_{\text{sizes}}\rangle \mapsto \langle \ell_{sz}, 0\rangle$
			$\forall i.\ \langle \ell_{sz}, i \cdot \text{sz}_{\text{int64\_t}}\rangle \mapsto \alpha(d_i)$

$\mathcal{R}_{\textsc{List}}$
$\phi = \textsc{List}\langle type\rangle(\vec{e})$
$\vec{e} = [e_1 \dots e_n]$
	`struct {`
	`size_t size;`
	`T* data;`
	`}`
		$\ell = \text{alloc(List}\langle type\rangle)$
		$\ell_{buf} = \text{alloc}(type[n])$
		$v \mapsto \langle \ell, 0\rangle$
			$\langle \ell, \delta_{\text{size}}\rangle \mapsto n$
			$\langle \ell, \delta_{\text{data}}\rangle \mapsto \langle \ell_{buf}, 0\rangle$
			$\forall i.\ \text{Process}(type, e_i, \langle \ell_{buf}, i \cdot \text{sz}_{type}\rangle)$

$\mathcal{R}_{\textsc{Optional}}$
$\phi = \textsc{Optional}\langle type\rangle(u)$
	`struct {`
	`bool has_val;`
	`T* val;`
	`}`
		$\ell = \text{alloc(Optional}\langle type\rangle)$
		if $u \neq \text{None}$:
		$\ell_{obj} = \text{alloc}(type)$
		$v \mapsto \langle \ell, 0\rangle$
			$\langle \ell, \delta_{\text{has\_val}}\rangle \mapsto (u \neq \text{None})$
			if $u \neq \text{None}$:
			$\langle \ell, \delta_{\text{val}}\rangle \mapsto \langle \ell_{obj}, 0\rangle$
			$\text{Process}(type, u, \langle \ell_{obj}, 0\rangle)$

For Scalar and String types, the corresponding context-update rule directly stores the value in the environment. For composite types like Tensor and List, the context-update rules perform *deep reconstruction*: allocating base objects for internal structures and linking them via pointers in the store. We use a helper function $\alpha(x)$ that returns x if x is a concrete constant, or $\perp$ if x is a symbolic expression~~. We use $\ell \in \mathcal{B}$ to denote a base object returned by allocation operations~~, where $\perp$ denotes *unknown-but-consistent* values fixed at runtime but not statically determinable.

The recursive helper function ~~$\mathrm{Process}(\phi_1, v, \ell)$~~ $\mathrm{Process}(\mathit{type}, v, \ell)$ populates the store at location ℓ with value v of type ~~ϕ_1. Here, δ_{field} denotes the byte offset of a structure field, and sz_T denotes the size of type T in bytes~~*type*. For example, the ~~$\mathit{Process}(\mathit{Tensor}, [d_1, \ldots, d_k], \ell)$~~ $\mathcal{R}_{\textsc{Tensor}}$ context-update rule is defined as:

$$
\begin{aligned}
&\mathbf{Process}(\texttt{Tensor}, [d_1, \ldots, d_k], \ell) : \\
&\quad \ell_{sz} = \mathrm{alloc}(\texttt{int}[k]) \\
&\quad \mathbb{S}[\langle \ell, \delta_{\texttt{sizes}} \rangle] = \langle \ell_{sz}, 0 \rangle; \ \mathbb{S}[\langle \ell, \delta_{\texttt{ndim}} \rangle] = k \\
&\quad \textbf{for } i = 1 \textbf{ to } k \textbf{ do } \mathbb{S}[\langle \ell_{sz}, (i-1) \cdot \mathrm{sz}_{\texttt{int}} \rangle] = \alpha(d_i)
\end{aligned}
$$

Example 3.2. Consider a `Tensor` parameter `query` with shape $[s_1, 40]$ (symbolic s_1, concrete 40). After applying the ~~update rule:~~ $\mathcal{R}_{\textsc{Tensor}}$ context-update rule:

$$
\begin{aligned}
&\mathbb{E} = \{\texttt{query} \mapsto \langle \ell, 0 \rangle\} \\
&\mathbb{S} = \{\langle \ell, \delta_{\texttt{sizes}} \rangle \mapsto \langle \ell_{sz}, 0 \rangle, \ \langle \ell_{sz}, 0 \rangle \mapsto \perp, \ \langle \ell_{sz}, \mathrm{sz}_{\texttt{int}} \rangle \mapsto 40\}
\end{aligned}
$$

The environment maps `query` to a pointer, and the store captures the internal structure with $\perp$ for symbolic dimension s_1 and 40 for the concrete dimension.

Completeness. The context-update rules cover all five core parameter types (Scalar, String, Tensor, List, Optional) in the PyTorch operator interface. Recursive applications of the context-update rules enable complex compositions ~~like~~such as ~~$\mathit{Optional}\langle \mathit{List}\langle \phi_1 \rangle \rangle$~~$\textsc{Optional}\langle \textsc{List} \rangle$, ensuring that the approach is generic across standard operator schemas.

3.3 ~~Identify Early Exit~~ Early-Exit Edge Identification

Production operator libraries manage execution status through compile-time constant integer status codes. ~~Based on the status code definitions in the operator library, we manually extract two setsas inputs to our identification algorithm~~We define two sets: $\mathcal{S}$ (success codes, e.g., $\{0\}$) and $\mathcal{E}$ (error codes, e.g., $\{-1\}$). The early-exit edge identification process consists of three steps: (1) constructing a Conditional Return-Value Flow Graph (CRVFG) to capture the relationship between ~~shape checker~~ shape-checker conditions and return values; (2) determining ~~whether each return value under its governing condition belongs to error states~~ error states through constant comparison; and (3) mapping identified error conditions to control-flow edges.

3.3.1 Conditional Return-Value Flow Graph Construction. The first step constructs a *Conditional Return-Value Flow Graph* (CRVFG) to capture the relationship between ~~shape checker~~ shape-checker conditions and return values. The CRVFG is a directed graph $G = (V, E)$ where:

- Each node $v \in V$ represents a definition ~~or a return instruction.In LLVM IR, definitions include phi, store, load~~of the return value (e.g., ~~call, and other value-producing instructions~~a `phi` node or a constant assignment).
- Each edge ~~$e = (v_s, v_d) \in E$~~ $e = (v_s, v_d, \varphi) \in E$ represents a data-flow dependency from v_s to v_d under branch predicate φ.

The CRVFG is constructed *on-demand* ~~starting from the return instruction, tracing backward through data-flow dependencies exclusively for the return value . During construction, phi instructions are split into multiple nodes according to the guard conditions on incoming CFG edges. For a phi node with n incoming operands, we create n~~

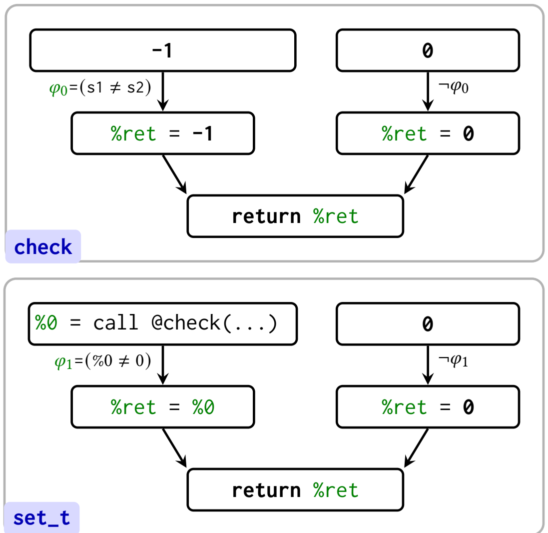


Fig. 9. ~~Return-Value Flow Graphs for check and set_t functions. For check: direct error return when $\varphi_0 = (s1 \neq s2)$ holds, the return value is $-1 \in \mathcal{E}$. For set_t: propagated error when $\varphi_1 = (\%0 \neq 0)$ holds, the error code from check flows to the return.~~ Return-Value Flow Graphs for check and set_t. In check, when $\varphi_0 = (\texttt{s1} \neq \texttt{s2})$ holds, the function returns $-1 \in \mathcal{E}$. In set_t, when $\varphi_1 = (\%0 \neq 0)$ holds, the error code returned by check flows to the function return.

Algorithm 1. Early-Exit Edge Identification

Require: Function F, Success set $\mathcal{S}$, Error set $\mathcal{E}$
Ensure: Set of early-exit edges E_{exit}

$E_{exit} \leftarrow \emptyset$
$inst \leftarrow \texttt{GetReturnInst}(F)$
$v_{ret} \leftarrow \texttt{GetReturnValue}(inst)$
$P_{flow} \leftarrow \texttt{BuildCRVFG}(F, v_{ret})$
for all $path \in P_{flow}$ **do**
  $\varphi \leftarrow \texttt{GetCondition}(path)$
  $v \leftarrow \texttt{GetSourceValue}(path)$
  if isError(v, φ) **then**
    $\varphi_{key} \leftarrow \texttt{GetKeyClause}(\varphi)$
    $B_{src} \leftarrow \texttt{GetBasicBlock}(\varphi_{key})$
    $B_{dst} \leftarrow \texttt{GetSuccessor}(B_{src}, \varphi_{key})$
    $\texttt{MarkEarlyExitEdge}(B_{src} \rightarrow B_{dst})$
    $E_{exit} \leftarrow E_{exit} \cup \{B_{src} \rightarrow B_{dst}\}$
  end if
end for
return E_{exit}

~~separate nodes, each associated with the condition guarding the corresponding control-flow path.~~ and *intraprocedurally*:

(1) **On-Demand:** We construct the graph only for functions containing shape checkers, avoiding overhead for irrelevant functions.
(2) **Return-Value Focus:** We exclusively track the value flow of return values, which in SSA-form IR are top-level variables, yielding compact graphs.

Figure 9 illustrates the CRVFGs for both the check and set_t functions in Code 1. In check, the condition $\varphi_0 = (\texttt{s1} \neq \texttt{s2})$ guards the path where the return value is $-1 \in \mathcal{E}$, directly identifying an ~~error return~~ early-exit path. In set_t, the condition $\varphi_1 = (\%0 \neq 0)$ guards the path where the error code from check flows to the return. The CRVFG thus characterizes the conditions guarding each value-flow path, enabling us to determine which paths lead to error states.

3.3.2 Error Status Determination. The second step determines whether a return value indicates an error state under a given condition.

Operator shape checkers fall into two categories based on the source of their return values: (1) *Direct-constant-return checkers* (e.g., the check function) that directly return a constant value~~(~~, typically an error code~~)~~, when validation fails; and (2) *Variable-return checkers* (e.g., the set_t function) that return a variable, typically the return value of another function call.

For a value-flow path with return value v and governing condition φ, we ~~determine it is an error~~ classify it as an early-exit path if and only if:

$$\text{isError}(v, \varphi) = \begin{cases} \text{true} & \text{if } \varphi \Rightarrow (v \notin \mathcal{S}) \lor \varphi \Rightarrow (v \in \mathcal{E}) \\ \text{false} & \text{otherwise} \end{cases} \tag{6}$$

Intuitively, ~~this means an error~~ an early-exit path is identified when ~~the condition~~ φ implies that ~~the return value~~ v is either not a success code or is an error code.

For *Direct-constant-return checkers*, ~~determining whether $\varphi \Rightarrow (v \notin \mathcal{S})$ or $\varphi \Rightarrow (v \in \mathcal{E})$ is straightforward:~~ since v is a constant, we directly test whether $v \notin \mathcal{S}$ or $v \in \mathcal{E}$ holds. For *Variable-return checkers*, ~~the condition~~ φ ~~only involves~~ contains comparisons against constants (e.g., $v = -1$ or $v \neq 0$), where the constants are typically error ~~codes~~ or success codes. ~~To determine whether $\varphi \Rightarrow (v \notin \mathcal{S})$ or $\varphi \Rightarrow (v \in \mathcal{E})$, we directly~~ We compute the possible value set of ~~the return value variable from the condition~~ v from φ, ~~and~~ then test whether all possible values are ~~not in~~ outside $\mathcal{S}$ or belong to $\mathcal{E}$. Both cases ~~involve only set membership~~ require only constant comparison and set-membership operations computable in $O(1)$ time, without invoking an SMT solver.

Figure 9 illustrates this ~~determination~~ process. For the `check` function, under condition $\varphi_0 = (\texttt{s1} \neq \texttt{s2})$, the return value is the constant -1. Since ~~$-1 \in \mathcal{E}$, we directly have $v \in \mathcal{E}$, and thus isError$(-1, \varphi_0)$ = true. For the~~ $-1 \in \mathcal{E}$, isError$(-1, \varphi_0)$ = true. For `set_t` function, under condition $\varphi_1 = (\%0 \neq 0)$, the return value is $\%0$~~(the return value of~~, which is returned by `check`~~)~~. From φ_1, we infer that the possible values of $\%0$ are $\mathbb{Z} \setminus \{0\}$. Since $\mathcal{S} = \{0\}$, all possible values are ~~not in~~ outside $\mathcal{S}$, so isError$(\%0, \varphi_1)$ = true .

3.3.3 *Early-Exit Edge Identification on CFG.*

The third step maps identified ~~error conditions to control-flow edges in the CFG~~ early-exit conditions to CFG edges, as presented in Algorithm 1. For each value-flow path in the CRVFG, where a path represents a complete flow from a source (~~either~~ a call instruction or ~~a~~ constant) to the return instruction, the algorithm extracts the guard condition φ ~~(conjunction of conditions along the path) and the~~ and source value v, then invokes isError(v, φ) to determine whether ~~this path leads to an error return~~the path is an early-exit path. If an ~~error~~ early-exit path is identified, the algorithm extracts the key clause φ_{key} from φ via `GetKeyClause`~~, which~~. This operation returns the rightmost sub-clause (e.g., $\varphi = (b_1 = b_2) \land (\texttt{s1} \neq \texttt{s2})$ yields $\varphi_{key} = (\texttt{s1} \neq \texttt{s2})$), because the rightmost clause determines the error status under short-circuit evaluation~~semantics~~. The algorithm ~~then~~ obtains the basic block B_{src} containing φ_{key} via `GetBasicBlock`(φ_{key}) and the successor block B_{dst} via `GetSuccessor`(B_{src}, φ_{key}), then marks the CFG edge $B_{src} \rightarrow B_{dst}$ as an early-exit edge.

3.3.4 ~~*Inter-Procedural*~~*Interprocedural Recursion.*

When a function containing shape checkers has ~~its~~ a return value derived from a callee~~function~~, ~~inter-procedural~~interprocedural recursion is required to ~~propagate error status information~~ identify early-exit paths across functions. ~~Our algorithm identifies two scenarios that~~ Two scenarios trigger recursive analysis of ~~callee functions~~a callee:

(1) If the source value v of a value-flow path is the return value of a call instruction, the callee ~~function~~ is recursively analyzed.

(2) If a clause in the guard condition φ checks whether the return value of a call instruction belongs to $\mathcal{E}$ or $\mathcal{S}$ (e.g., $\varphi = (\%0 \neq 0)$ where $\%0$ is a call return), the callee ~~function~~ is recursively analyzed. This ~~scenario~~ case corresponds to common ~~status propagation~~ status-propagation patterns in production code (e.g., `if (x == ERROR) return ERROR`), where `x` is the return value of ~~one calleefunction~~a callee.

3.4 ~~Inter-Procedural~~Interprocedural Constant Propagation for Host-Side Programs

The propagation stage takes the initial calling context $\mathbb{C}_{init}$ from §3.2 and the early-exit edges E_{exit} from §3.3 as inputs, and propagates ~~constant information~~ constants through the host-side program to kernel launch sites. The analysis maintains a symbolic environment $\mathbb{E}$ and a flow-sensitive memory store $\mathbb{S}$, performing value propagation ~~on value domain as~~ over the domain defined in Table 1.

Initialization. The analysis begins with $\mathbb{C}_{init} = (\mathbb{E}, \mathbb{S})$ constructed from the Graph IR ~~node~~metadata, which provides concrete values for tensor dimensions and operator parameters. These ~~initial constants serve as seeds for~~ constants seed propagation: when the analysis ~~encounters a load~~ loads from a memory location initialized with a concrete value, that value flows into the corresponding virtual register.

Propagation and Edge Liveness. During forward propagation, conditional branches ~~are analyzed to~~ determine edge liveness. If a branch condition evaluates to a constant, the non-taken edge is marked `Inactive`; otherwise, both ~~branches~~ edges remain `Active`. At control-flow merge points, ~~the~~ incoming stores from all `Active` predecessors are combined using standard lattice ~~join operations~~joins.

Early-Exit Integration. A key precision enhancement comes from integrating early-exit edge information. Any CFG edge in E_{exit} is treated as `Inactive` during context merging. This excludes ~~error-handling~~ early-exit paths from the analysis state, preventing the pollution of propagated constants by imprecise values from unreachable branches. For example, if a shape checker returns an error code when dimensions mismatch, but the initial context guarantees matching dimensions, the error path is pruned and its state does not affect downstream analysis.

3.5 Kernel Specialization and Library Pruning

Based on the context derived from ~~the inter-procedural analysis~~interprocedural constant propagation, TilingInfer performs two key optimizations: ~~generic~~ schedule-generic kernel specialization and pruning of redundant template instantiations ~~for manually specialized~~ from schedule-fixed operator libraries.

For ~~generic~~ schedule-generic kernels, TilingInfer substitutes ~~the variable inside the kernel~~ kernel variables with the concrete schedule parameters identified ~~from the analysis on~~ by the host-side ~~program~~analysis. ~~Then TilingInfer employs~~TilingInfer then applies a sequence of optimization passes, specifically `IPSCCP`, `Loop Unrolling`, and `CFG Simplification`. These passes exploit the newly introduced constants to fold ~~selar~~ computations and simplify control structures, generating a highly optimized kernel binary.

For ~~manually specialized~~ schedule-fixed libraries, TilingInfer reduces code size by leveraging path liveness information from the host analysis. During constant propagation, we identify `Dead Paths`, i.e., execution branches where guard conditions evaluate to false based on deterministic constants. If a specific template function instance is invoked only on these dead paths, TilingInfer determines that it is unreachable at runtime and removes the instance entirely. This selective pruning ensures that only potentially active operator instances are retained in the final binary.

4 Evaluation

Our evaluation addresses the following research questions: **RQ1:** Can TilingInfer enhance performance across diverse operators and ~~models based on generic kernels~~neural networks that are based on schedule-generic expert-written libraries? **RQ2:** Can TilingInfer effectively infer more invariant schedule parameters with reasonable compilation overhead? **RQ3:** Can TilingInfer effectively eliminate redundant code ~~for manually specialized GPU~~ from schedule-fixed

Table 3. Overview of dynamic shape operator benchmarks. **Symbols:** T: total tokens, B: batch size, P: page size, N_P: number of KV-cache pages, M_P: maximum pages per request, N_Q/N_{KV}: query/key–value head counts, D: model dimension, D_{head}: head dimension (D/N_Q), K: matrix reduction dimension, C: channels, E: experts, and H/W: image height/width. **Aux. Input**: indexing or geometric metadata. **Configurations:** Unless otherwise stated, we fix static dimensions based on representative models: $D = 4096$, $D_{head} = 128$, $N_Q = 32$, $N_{KV} = 32$ for Llama2-7B; $E = 8$ for Mixture-of-Experts (MoE) layer; $C = 512$ for CV (e.g., ResNet-50); and $D = 128$ for RecSys. **Note:** Operators labeled **MM**, **FFN**, and **RMS** share identical kernel implementations across Prefill and Decode phases, differing only in the leading dynamic dimension (T vs. B). **Conv2d** is lowered to `im2col` followed by MatMul and is therefore reported as **MM**. Unless otherwise stated, each operator-level result is the geometric mean over five shapes sampled from the listed dynamic range; all methods use the same samples.

Domain	Phase / Scenario	Operator	Abbr.	Main Layout	Aux. Input	Dynamic Range
LLM *Llama 2* *Qwen 2* *DeepSeek V3*	Prefill Phase (Ragged Input)	FlashAttention	**P-Attn**	$Q : [T, N_Q, D_{head}]$ $K, V : [T, N_{KV}, D_{head}]$	`Offsets[B+1]`	$T \in [256, 8k]$ $B \in [1, 8]$
		FusedFFN	**FFN**	$[T, D]$	-	$T \in [256, 8k]$
		RmsNorm	**RMS**	$[T, D]$	-	$T \in [256, 8k]$
		MatMul	**MM**	$[T, K]$	-	$T \in [256, 8k]$
	Decode Phase (Paged KV)	PagedAttention	**D-Attn**	$Q : [B, 1, N_Q, D_{head}]$ $K, V : [N_P, P, N_{KV}, D_{head}]$	`BlockTbl[B,` M_P`]` `Offsets[B+1]`	$T \in [256, 8k]$ $B \in [1, 8]$
		FusedFFN	**FFN**	$[B, D]$	-	$B \in [1, 8]$
		RmsNorm	**RMS**	$[B, D]$	-	$B \in [1, 8]$
		MatMul	**MM**	$[B, K]$	-	$B \in [1, 8]$
	MoE Layer (Sparse Activation)	MoEGating	**Gate**	$[T, E]$	-	$T \in [256, 8k]$
		GroupedMatMul	**GMM**	$[T, D]$	`GroupOffsets[E+1]`	$T \in [256, 8k]$
		Sinkhorn	**Sink**	$[T, E]$	-	$E \in [8, 64]$
		MoETokenPermute	**Perm**	$[T, D]$	`Indices[T]`	$T \in [256, 8k]$
CV *ResNet,* *YOLOv5*	General Inference (Standard Batch)	Conv2d (im2col)	**MM**	$[B, C, H, W]$	-	$B \in [1, 8]$
		GroupNorm	**GN**	$[B, C, H, W]$	-	$B \in [1, 8]$
		AvgPool	**Pool**	$[B, C, D, H, W]$	-	$B \in [1, 8]$
		Elementwise Pow	**Pow**	$[B, C, H, W]$	-	$B \in [1, 8]$
	Object Detection (Dyn. Resolution)	Upsample	**Upsample**	$[B, C, H, W]$	-	$H, W \in [224, 1024]$
		BatchNorm	**BN**	$[B, C, H, W]$	-	$B \in [1, 8]$
RecSys	Feature Interaction (Sparse Update)	ScatterElements	**Scat**	$[T, D]$	`Indices[T]`	$T \in [128, 1024]$

expert-written operator libraries? Beyond these questions, we evaluate detailed performance metrics and improvements under different shape configurations.

4.1 Experimental Setup

Hardware and Software Platforms. We use two platforms~~:~~. (1) The **NPU ~~Platform~~platform** ~~—~~consists of an Ascend 910B1 NPU (64 GB HBM)~~ with~~, an Intel Core i7-8700 CPU ~~and~~, and 64 GB DRAM, running CANN 8.0.RC1, LLVM 15.0.5, and PyTorch 2.6.0. The end-to-end model experiments additionally use Transformers 4.57.6 and TorchRec 1.2.0. All end-to-end model-inference experiments enable NPU Graph replay for every compared method, so the methods use the same graph-execution setting and repeated CPU launch overhead is suppressed. Each reported metric is averaged over 90 measured runs after 10 warm-up iterations. ~~(reported metrics are averaged over 90 runs after 10 warm-up iterations);~~ (2) The **GPU ~~Platform~~platform** ~~—~~consists of an NVIDIA RTX 3090 (24 GB VRAM) ~~with~~and an Intel Xeon Gold 6348 CPU, running CUDA 12.2, PyTorch 2.6.0, vLLM 0.9.1, and FlashInfer v0.2.

Benchmarks. For the **NPU platform**, we evaluate individual operator latency and end-to-end model inference. The operator benchmark covers operators from LLMs, ~~Computer Vision~~computer vision, and recommendation models, with

Table 4. End-to-end model benchmark configurations. B is the number of concurrent requests, S is the HSTU interaction-sequence or LLM prompt length, and L_{KV} is the number of cached tokens visible to each decode request.

Scenario	Deployment stack	Fixed configuration	Dynamic Range in Figure 11	Shape in Figure 12
DeepFM	`torch_npu` + TorchRec	20 sparse fields, 13 dense features, $D_{emb} = 64$, 10 dense blocks	$B \in \{4, 16, 32, 64, 128\}$	$B = 64$
HSTU	`torch_npu` + TorchRec	$D = 512$, $N_H = 8$, $D_H = 64$, 8 HSTU blocks; no interaction-sequence cache	$B = 2,\ S \in \{256, 512, 1024, 2048, 4096\}$	$B = 2,\ S = 256$
Llama 3.2 1B Prefill	`torch_npu` + Transformers	$D = 2048$, $N_H = 32$, $N_{KV} = 8$, $D_H = 64$, 16 decoder blocks	$B = 2,\ S \in \{256, 512, 1024, 2048, 4096\}$	$B = 2,\ S = 256$
Llama 3.2 1B Decode	`torch_npu` + Transformers	one-token query, $D = 2048$, $N_H = 32$, $N_{KV} = 8$, $D_H = 64$, paged KV cache	$B = 2,\ L_{KV} \in \{256, 512, 1024, 2048, 4096\}$	$B = 2,\ L_{KV} = 256$
Qwen3.5-0.8B Prefill	`torch_npu` + Transformers	$D = 1024$, $N_H = 8$, $N_{KV} = 2$, $D_H = 256$, 24 language-model layers	$B = 2,\ S \in \{256, 512, 1024, 2048, 4096\}$	$B = 2,\ S = 256$
Qwen3.5-0.8B Decode	`torch_npu` + Transformers	one-token query, $D = 1024$, $N_H = 8$, $N_{KV} = 2$, $D_H = 256$, paged KV cache	$B = 2,\ L_{KV} \in \{256, 512, 1024, 2048, 4096\}$	$B = 2,\ L_{KV} = 256$

emphasis on complex fused operators such as `FlashAttention`, `FeedForward`, and `MatMul` that ~~feature intensive~~have substantial workloads and numerous schedule parameters. Table 3 summarizes the categorization and configurations of the evaluated operators. For end-to-end evaluation, we ~~use Qwen2~~ evaluate six scenarios comprising DeepFM recommendation inference [12], HSTU generative recommendation [42], Llama 3.2 1B prefill and decode, and Qwen3.5-0.8B prefill and decode. Table 4 provides the fixed model configurations, dynamic-shape range, and representative shapes.

For the **GPU platform**, we target FlashInfer [40], a state-of-the-art attention kernel library used in vLLM, and evaluate binary size reduction for Llama and Qwen model families.

Implementation Details. All evaluations for both operators and models are conducted using PyTorch 2.6.0. Dynamic dimensions are explicitly marked using the `Dynamic` API, and we utilize PyTorch Dynamo with dynamic compilation enabled. For the operator benchmark, we compare **TilingInfer** against the following baselines: **Baseline**, standard offline compilation with LLVM passes ~~without kernel specialzation~~but without kernel specialization; **LLVM-spec**, which performs constant propagation within standard LLVM~~, but without early-exit identification and TilingInfer's inter-procedural~~but without early-exit identification or TilingInfer's interprocedural constant propagation. The derived schedule parameters are then used to specialize kernels ~~same as TilingInfer~~ in the same way as in TilingInfer; **Optimal**, ~~serves as an oracle reference. Optimal is computed by specializing the kernel with~~ a per-shape full-specialization oracle that specializes each kernel with all constant schedule parameters derived for each individual shape configuration. ~~Since this approach requires enumerating all possible shapes at compile time, it is infeasible for practical deployment~~It is unsuitable for production inference because dynamic inputs require per-shape recompilation, incurring prohibitive compilation overhead.

4.2 Performance Results on Ascend

4.2.1 Operator Performance Analysis. Figure 10 presents the normalized speedup of the evaluated operators across three methods: **LLVM-spec**, **TilingInfer**, and ~~the~~ **Optimal**. All results are normalized to the **Baseline** (standard offline compilation).

Overall, **TilingInfer** achieves a geometric mean speedup of **1.06×** across all operators, substantially outperforming the **LLVM-spec** baseline (1.01×). Crucially, the speedup ratio of **TilingInfer** is close to that of **Optimal** (only 1% smaller on average). This demonstrates that **TilingInfer** successfully specializes the majority of ~~kernel~~ schedule parameters, leaving only a few residual parameters that have negligible impact on performance. These results validate that **TilingInfer** achieves effective automatic kernel specialization for NPU operators.

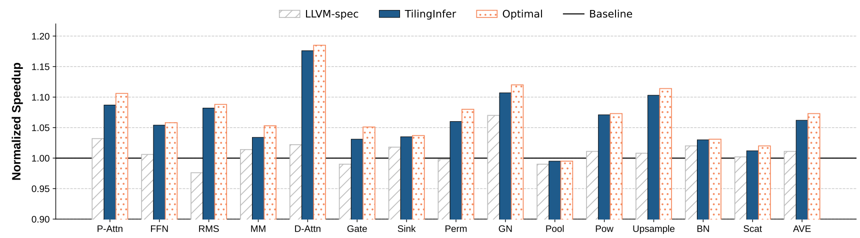


Fig. 10. Overall normalized performance of **TilingInfer** compared to compiler baselines

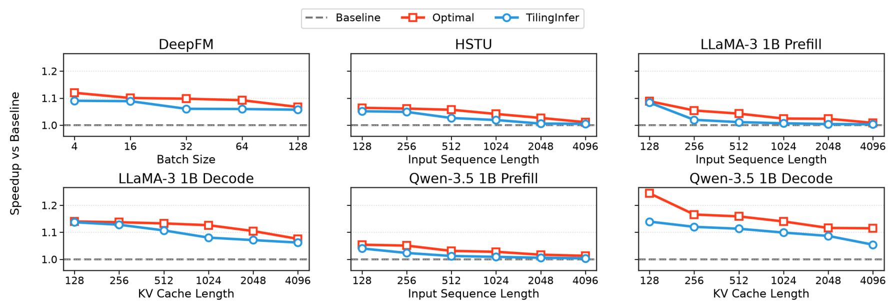


Fig. 11. End-to-end speedup versus runtime shape for six representative ML inference scenarios.

~~We observe distinct performance behaviors across operator categories: Attention mechanisms (P-Attn, D-Attn) show the most significant gains, 1.09× for P-Attn and 1.18× for D-Attn because these kernels have a large number of schedule parameters.~~ We observe distinct performance behaviors across operator categories. **Attention operators** show the largest gains: **1.08**× for P-Attn and **1.17**× for D-Attn, because these schedule-generic kernels expose many constant schedule parameters. RMS and GN also benefit notably (**1.08**× and **1.11**×), as constant specialization eliminates redundant boundary checks and simplifies division operations.

4.2.2 *Representative End-to-End ~~Model Performance Analysis~~Inference Scenarios.* ~~The end-to-end model inference performance is depicted in , showing normalized speedup for Llama-3.2-3B and Qwen2.5-1.5B across varying sequence lengths and batch sizes (BS=1, 2, 4) during both Prefill and Decode phases. Additionally,~~

~~presents the operator latency breakdown to illustrate the contribution of different operators to the total inference time.~~

The shape configurations of the six representative inference scenarios are summarized in Table 4, and their operator calls are classified in Table 5. We divide the calls into seven categories. MatMul/GMM, Attention, and Norm invoke CANN kernels from a schedule-generic implementation and together account for approximately 40%–94% of model-inference latency. The other four categories use kernels generated by TVM [6], an AI compiler that provides a Python-embedded DSL for describing operator implementations. These kernels have simple schedules, with schedule parameters already specialized during code generation, so TilingInfer provides no further speedup for them.

~~As shown in , achieves an average speedup of 3% across various shape configurations for both Prefill and Decode phases. The speedup is more pronounced for shorter sequences, but~~ Figure 11 shows that TilingInfer's end-to-end speedup generally decreases as the ~~input length and the number of generated tokens increase. This trend is consistent across both phases.~~ runtime shape grows because tensor computation occupies an increasing fraction of execution time. DeepFM and the two decode scenarios obtain the larger gains, averaging approximately 1.10×. These scenarios are

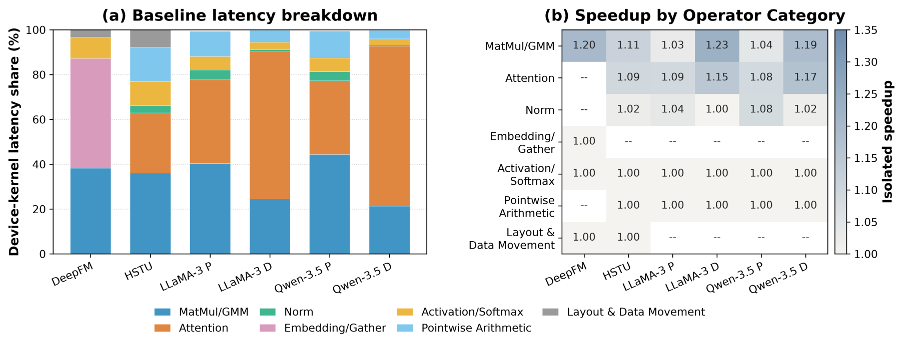


Fig. 12. Device-kernel latency analysis for six inference scenarios at representative shapes.

relatively more memory-intensive and use smaller, shorter-running device-kernel workloads, so the scalar overhead removed by specialization accounts for a larger fraction of latency. In contrast, the compute-intensive HSTU and prefill scenarios average approximately 1.03× because their device kernels perform more tensor computation. Relative to Optimal's per-shape full specialization, TilingInfer achieves comparable geometric-mean speedups—1.05× versus 1.08× across all six scenarios and 1.10× versus 1.14× across the two decode scenarios; although faster, Optimal recompiles the entire operator for every shape, taking tens of minutes each and thus being impractical in production.

~~We make two key observations. First, Speedup Variation with Sequence Length, the end-to-end speedup is more significant for small sequences where scalar overhead is proportionally larger ; as sequence length increases, computational workload dominates, decreasing the benefit. Second, Dominance of MatMul, MatMul accounts for the majority of inference time; although achieves significant speedups for FlashAttention and normalization, these contribute less to overall latency, resulting in modest end-to-end improvement~~

Table 5. Operator categories, implementation sources, and TilingInfer outcomes for the six representative models.

Category	Operators	Implementation source / outcome
MatMul/GMM	Matrix Multiplication (MM), Batched Matrix Multiplication (BMM), Grouped Matrix Multiplication (GMM)	CANN operator library; TilingInfer specialization benefits
Attention	P-Attn, D-Attn	
Norm	RMSNorm, LayerNorm, GroupNorm	
Embedding/Gather	Embedding, Gather	AI-compiler-generated kernels (TVM Python DSL); no TilingInfer specialization benefit
Activation/Softmax	ReLU, SiLU/Swish, GELU, Sigmoid, Tanh, Softmax, Reduce/Mean	
Pointwise Arithmetic	Add, Sub, Mul, Div, Pow, Rsqrt, Maximum/Minimum, Zero	
Layout & Data Movement	Concat, Slice, Transpose, Cast, Copy/TensorMove, Reshape/AsStrided	

Figure 12 attributes the model-level gains primarily to Attention and MatMul/GMM. In particular, the MatMul calls in DeepFM and the LLM decode phase have relatively small computation sizes, so specialization removes a larger fraction of their tiling and control overhead and produces higher operator speedups. Attention supplies the other major source of improvement, especially in the decode phase. By contrast, the four AI-compiler-generated categories remain at 1.00×, confirming that TilingInfer's benefits are limited to expert-written operators.

4.2.3 ~~Comparison between TilingInfer and specialization based on Standard LLVM~~ Comparison with Standard LLVM Specialization. Table 6 shows the trade-off between compilation overhead and optimization effects. **LLVM-spec** incurs minimal overhead (less than 0.60 s) due to its ~~inprecise~~ imprecise analysis. In contrast, **TilingInfer** ~~exhibits a moderate increase (0.24s to 2.70s)~~ adds 0.24–2.70 s due to field-sensitive analysis. Given that this compilation is a one-off cost during deployment, we consider this overhead acceptable, as it enables the backend compiler to generate considerably more efficient binaries (1.06× speedup). Moreover, **TilingInfer** ~~avoids employing path-sensitive analysis through early-exit path identification~~avoids path-sensitive analysis by using early-exit edge identification, keeping compilation overhead acceptable.

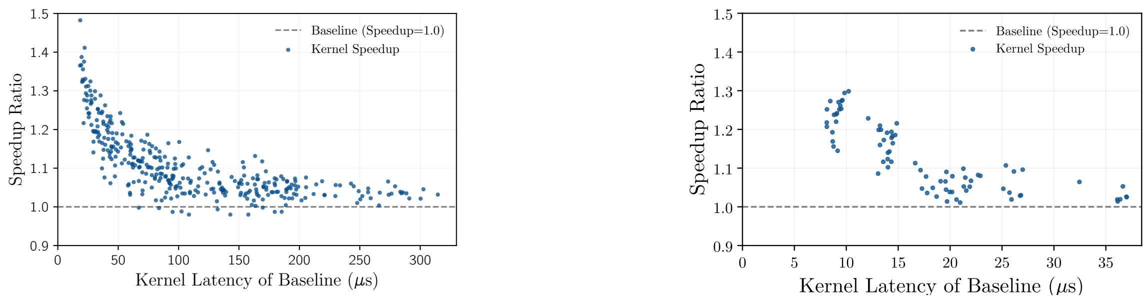
Table 6. Detailed comparison of compilation overhead and parameter specialization. **Total Params** indicates the number of schedule parameters defined in the operator program. **Time** is total compilation time in seconds. **Diff** is the time increment relative to Baseline. **Derived #** is the count of invariant schedule parameters inferred by the compiler.

Operator		P-Attn	D-Attn	MM	FFN	RMS	Gate	Sink	Perm	GN	Pool	Pow	Upsample	BN	Scat
Total Params		**264**	**389**	**72**	**123**	**15**	**60**	**20**	**42**	**23**	**17**	**10**	**143**	**22**	**20**
Baseline	Time (s)	24.50	19.80	42.10	4.52	3.12	1.85	5.10	1.24	6.15	1.35	0.82	2.95	3.30	1.15
LLVM-Spec	Time (s)	25.10	20.15	42.45	4.65	3.20	1.92	5.30	1.30	6.40	1.40	0.88	3.10	3.35	1.20
	Diff	(+0.60)	(+0.35)	(+0.35)	(+0.13)	(+0.08)	(+0.07)	(+0.20)	(+0.06)	(+0.25)	(+0.05)	(+0.06)	(+0.15)	(+0.05)	(+0.05)
	Derived #	4	3	3	2	1	2	0	0	1	0	0	3	2	1
TilingInfer	Time (s)	27.20	21.05	42.95	5.42	3.58	2.25	6.95	1.65	7.25	1.62	1.15	3.55	3.54	1.52
	Diff	(+2.70)	(+1.25)	(+0.85)	(+0.90)	(+0.46)	(+0.40)	(+1.85)	(+0.41)	(+1.10)	(+0.27)	(+0.33)	(+0.60)	(+0.24)	(+0.37)
	Derived #	242	352	60	110	11	30	13	28	17	15	5	82	15	14

4.3 Device-Level Attribution

Table 7. Performance comparison: Speedup and detailed hardware metrics across **Baseline** (B), **LLVM-spec** (L), and **TilingInfer** (T). Best values are bolded. Reflecting our goal to maximize time spent on tensor computation units (Cube and Vector), ↓ denotes lower is better for metrics not directly tied to tensor computation. ↑ denotes higher is better for speedup ratio.

Op	**Speedup (↑)**			**I-Cache Miss (%) (↓)**			**Scalar Ratio (%) (↓)**			**Mem Read Ratio (%) (↓)**			**Mem Write Ratio (%) (↓)**			**Binary Size (KB) (↓)**		
	B	**L**	**T**	**B**	**L**	**T**	**B**	**L**	**T**	**B**	**L**	**T**	**B**	**L**	**T**	**B**	**L**	**T**
P-Attn	1.00	1.03	**1.08**	16.40	16.30	**11.39**	91.23	**90.29**	90.33	**8.26**	8.30	8.95	3.94	**3.81**	4.11	363.81	360.76	**310.12**
FFN	1.00	1.01	**1.05**	3.89	4.12	**0.00**	84.72	84.52	**83.60**	6.42	7.24	**5.49**	**0.10**	0.11	0.11	68.79	68.89	**54.91**
RMS	1.00	0.98	**1.08**	**3.00**	3.01	3.36	97.89	97.82	**88.57**	6.83	7.27	**6.72**	**2.90**	3.02	3.13	36.32	36.32	**33.23**
MM	1.00	1.01	**1.03**	**3.12**	3.42	3.41	84.70	84.74	**84.68**	**42.66**	47.10	45.95	0.00	0.00	0.00	101.38	101.26	**100.84**
D-Attn	1.00	1.02	**1.17**	4.45	5.51	**0.00**	77.98	77.08	**74.43**	6.42	6.44	**4.91**	2.46	**1.60**	1.79	58.30	58.59	**45.05**
Gate	1.00	0.99	**1.03**	0.00	0.00	0.00	**96.98**	**96.98**	96.99	25.00	24.82	**16.43**	1.17	**1.05**	1.36	17.10	**17.09**	**17.09**
Sink	1.00	1.02	**1.03**	0.00	0.00	0.00	52.46	52.96	**50.34**	4.41	4.68	**4.34**	18.60	18.14	**17.88**	47.54	47.56	**47.11**
Perm	1.00	1.00	**1.08**	0.00	0.00	0.00	**92.34**	92.58	92.62	10.55	**9.31**	10.52	5.84	6.25	**5.69**	24.02	24.14	**14.40**
GN	1.00	1.07	**1.11**	0.80	1.20	**0.00**	99.10	98.94	**98.90**	24.98	**18.46**	18.83	14.20	**9.29**	9.31	220.76	220.56	**193.80**
Pool	1.00	0.99	0.99	0.00	0.00	0.00	97.27	97.21	**97.14**	45.01	45.01	45.01	0.00	0.00	0.00	13.35	13.35	**13.24**
Pow	1.00	1.01	**1.07**	0.00	0.00	0.00	59.45	57.60	**53.11**	44.64	45.48	40.16	2.72	2.80	3.00	24.58	24.58	24.31
Upsample	1.00	1.00	**1.10**	9.83	9.98	**0.00**	99.50	99.51	**98.42**	16.45	17.38	**13.87**	**1.76**	2.08	2.15	56.84	56.84	**37.54**
BN	1.00	1.01	**1.03**	0.00	0.00	0.00	**96.60**	96.69	96.82	64.86	66.16	**60.02**	22.94	**22.13**	22.44	15.73	15.69	**15.61**
Scat	1.00	1.00	**1.01**	24.38	23.42	**22.61**	**97.39**	98.06	97.93	5.44	4.36	**1.41**	**0.90**	1.29	1.12	24.11	23.99	**23.40**



(a) D-Attn

(b) MM

Fig. 13. Kernel speedup versus Baseline device-kernel latency for D-Attn and MM. Blue points report the speedup of TilingInfer relative to Baseline.

~~To gain deeper insight into the sources of performance gains demonstrated in Section 4.2, we analyze low-level hardware performance metrics collected during operator execution.~~ Table 7 ~~details the Speedup alongside key architectural metrics: Instruction Cache (I-Cache) Miss Rate, Scalar Instruction Ratio, Memory Access (Read and Write) Ratios, and the final compiled Binary Size. The Scalar Ratio, Memory Read Ratio, and Memory Write Ratio quantify the proportion of execution cycles of corresponding hardware units (scalar units, memory read/write units) relative to the total kernel execution cycles. Comparing TilingInfer (T) against the Baseline (B) and LLVM-spec (L), we attribute the improvements to two primary factors: reduced instruction fetch overhead and simplified runtime calculation.~~

~~Reduction in Binary Size and I-Cache Misses. By propagating runtime invariants, TilingInfer enables the backend compiler to aggressively perform Dead Code Elimination (DCE) and simplify control flow graphs. This leads to significant binary size reductions: 34.0% for Upsample, 22.7% for D-Attn, 20.2% for FFN, and 14.8% for P-Attn. Consequently, the I-Cache Miss rate drops substantially: Upsample (9.83% → 0.12%), D-Attn (4.45% → 0.18%), FFN (3.89% → 0.15%), and P-Attn (16.40% → 11.39%). These improvements in instruction fetch efficiency contribute to the observed speedups.~~

~~Optimization of Computational Intensity and Memory Schedule. TilingInfer substantially reduces the Scalar Ratio for operators like RMS (97.9% → 88.6%) and Pow (59.5% → 53.1%), confirming that resolving loop bounds and offsets at compile time effectively converts runtime scalar calculations into immediate operands. The Memory Read Ratio~~

~~also improves, dropping from 25.0% to 16.4% for Gate and from 25.0% to 18.8% for GN. This reduction is primarily due to the decreased number of parameters read from device memory, as well as the compiler's ability to merge multiple memory accesses when loop iteration counts are determined at compile time. Furthermore, optimizations such as loop unrolling can facilitate instruction scheduling and effectively hide memory access latency. These factors collectively contribute to the speedups observed in operators such as D-Attn, Pow, and RMS.~~

~~Insensitive Operators. Some operators, such as Pool and Scat, show negligible speedups (1.00×–1.01×). For Pool, the Baseline binary size is already minimal (13.35KB) with 0.08% I-Cache miss rate, leaving no room for control flow simplification. For Scat, the overhead of scalar instructions and instruction fetching is minimal compared to memory-bound operations. Optimizations in the instruction stream do not translate into visible latency reductions when the bottleneck lies in memory or arithmetic units.~~

~~Impact of Workload Scale. The original varying-KV-length and latency-distribution results show that speedup decreases towards 1.0 as workload increases. This implies that performance benefits from kernel specialization are relatively fixed in absolute time. Consequently, this fixed reduction yields significant speedup ratios for small-scale kernels but becomes negligible for heavy workloads dominated by computation and memory access.~~

Table 7 and Figure 13 show that TilingInfer does not produce uniform device-level gains; benefiting kernels fall into two mechanism-driven classes, whereas a third class is largely insensitive. For Upsample, D-Attn, FFN, and P-Attn, gains mainly arise because invariant propagation exposes constant branches and loop structure, enabling dead-code elimination and control-flow simplification; the resulting smaller instruction footprint reduces instruction-fetch and I-Cache pressure. A second class, including RMS, Pow, Gate, and GN, benefits primarily from simplified runtime calculation and memory scheduling. Resolving loop bounds, offsets, and schedule parameters removes scalar loads and address or control computations, exposes immediate operands, and enables access merging, loop unrolling, and more effective instruction scheduling. In contrast, Pool and Scat are largely insensitive: Pool is already compact with little instruction-fetch overhead, whereas Scat is dominated by memory work, so instruction-stream simplification does not shorten the critical path. Across all three classes, specialization mainly removes a roughly fixed amount of control and scheduling overhead. Its relative effect is therefore strongest for short-running kernels and diminishes as tensor computation or memory traffic dominates. The key determinant is not whether a parameter is static, but whether resolving it eliminates work on the kernel's performance-critical path.

4.4 Sensitivity to Resolved Schedule Parameters

This section analyzes whether specializing different groups of schedule parameters yields different benefits: can one group provide a substantially larger gain than the others?

For D-Attn, $\text{Query}_{b,h}\text{Key}^{\mathsf{T}}_{b,h}$ multiplies the Query row of request b and attention head h by its cached Key rows, while $\mathbf{T}$ stores each request's KV length. Its 34 schedule parameters form Control logic (6), Per-core tile dimensions/addressing (14), and Core-loop scheduling/memory resources (14). Control logic covers the branches and loop bounds that select how Key data are loaded. Per-core tile dimensions/addressing specifies how much work each core owns and maps tile indices to tensor addresses. Core-loop scheduling/memory resources controls how tiles are assigned to active cores and how on-chip buffers are allocated and reused inside the core loop. Figure 14(b) lists seven representative examples. In panel (c), T_N and K_c split Key rows and reduction columns into tiles, while M_c and N_c distribute the tiles to cores. TileOffset uses these extents and $\mathbf{T}$ to recover (b, h, n_0, k_0), the tile's starting coordinate. The schedule parameters T_N, P, and T_D select one page-aligned load or several narrow page-table loads, while n_K selects the Key buffer. BMM then loads the matching Query slice and computes the tile.

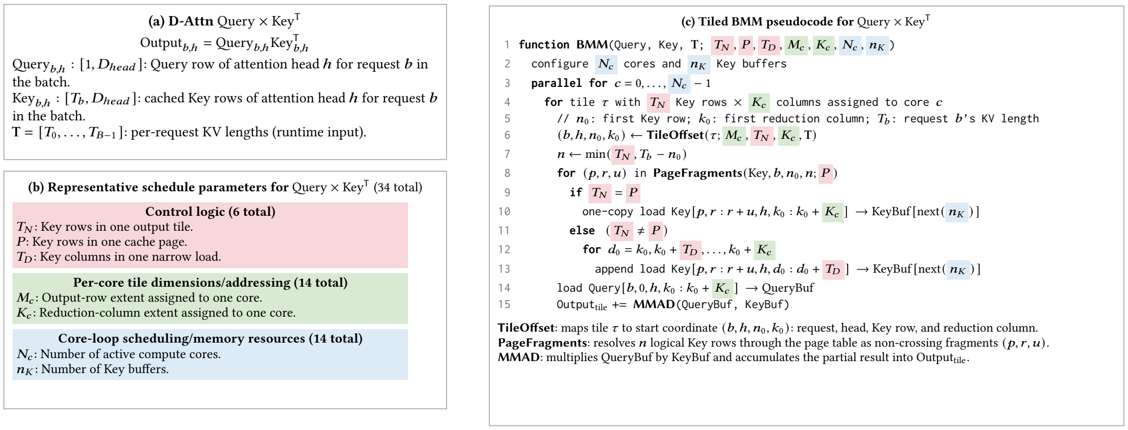


Fig. 14. Tiled D-Attn BMM. Panel (b) lists seven representative schedule parameters; the three groups contain 6, 14, and 14 schedule parameters in total. Matching backgrounds in panel (c) mark their use sites and correspond to the groups evaluated in Figure 15.

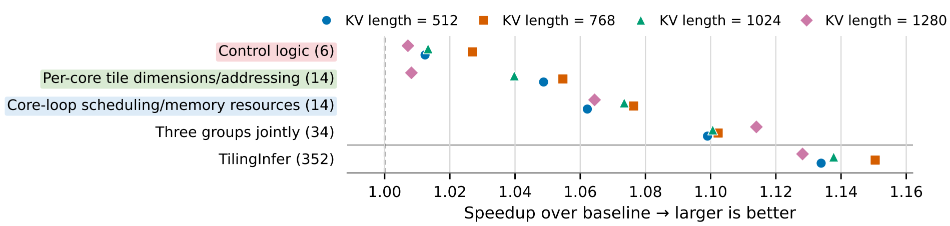


Fig. 15. Sensitivity to resolved schedule parameters across runtime shapes. Numbers in parentheses give the number of specialized schedule parameters. Each point shows the speedup at one KV length; the first three label backgrounds match the groups in Figure 14.

Overall, Figure 15 shows that specialization value depends more on a parameter's use site than on parameter count. Across KV lengths 512–1280, the 14 Core-loop scheduling/memory-resource parameters achieve the largest isolated geometric-mean speedup, 1.07×, by configuring per-core work and Key-buffer resources on the inner BMM path. The 14 Per-core tile dimension/addressing parameters reach 1.04×, while the 6 Control-logic parameters reach 1.02×. Thus, count alone poorly predicts specialization value.

The three isolated gains sum to 12.2%, whereas jointly specializing all 34 parameters yields 1.10× (10.4%), or 85.0% of that sum, indicating interaction and diminishing returns. TilingInfer specializes 352 constant schedule parameters and reaches 1.14×. Although the 34 Query × Key$^{\top}$ parameters comprise only 9.7% of these parameters, they recover 75.0% of the full gain; the remaining 318 add only 3.4 percentage points. Because these 34 parameters control D-Attn's core BMM in Figure 14(c), specializing its hot-path address and control flow is more valuable than resolving constants on colder paths. These conclusions apply to the evaluated D-Attn implementation and shapes.

4.5 Binary-Size Trade-off

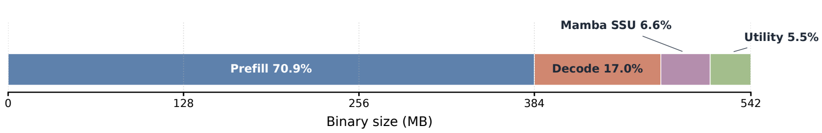


Fig. 16. Full AOT archive breakdown.

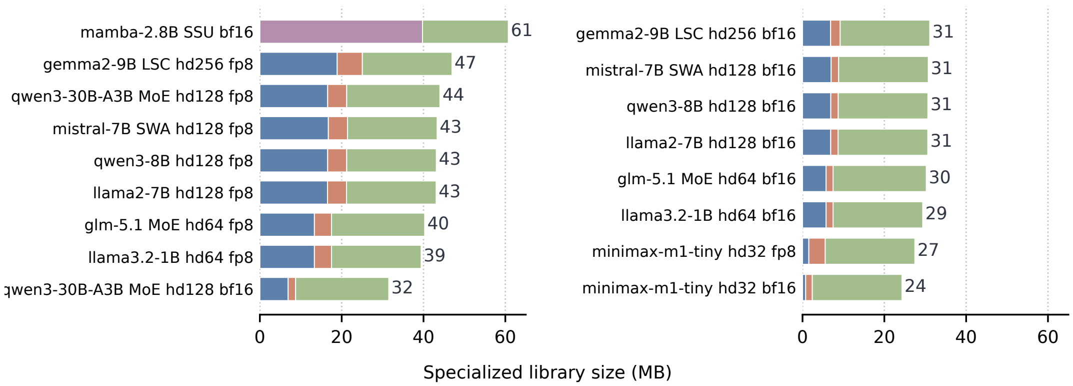


Fig. 17. Per-model FlashInfer footprint.

~~We adapted TilingInfer to eliminate redundant template instantiations never called at runtime. We targeted FlashInfer~~ [40]~~, a state-of-the-art attention kernel library widely used in LLM serving systems like vLLM and SGLang. The results of this optimization are presented in the original GPU binary-reduction table. We evaluated the binary size reduction for different LLM families (Llama and Qwen) with various model sizes. TilingInfer demonstrates remarkable effectiveness in eliminating redundant code, reducing the binary size of the FlashInfer library from approximately 520MB to roughly 20MB. On average, TilingInfer achieves a 96.2% reduction in binary size. This substantial reduction in library footprint benefits cold-start-sensitive scenarios such as serverless ML inference, where loading large operator libraries into GPU memory constitutes a major portion of initialization latency. Other DSA architectures with AoT-compiled operator libraries that suffer from similar code-bloat issues could also benefit from TilingInfer.~~

We apply path-liveness information to FlashInfer and retain the shared objects reachable by each of 17 model variants, including standard, FP8 KV-cache, MoE, sliding-window, and logits-soft-cap attention as well as Mamba Selective-State-Update (SSU) kernels. The universal AOT archive contains 247 shared objects and occupies 542 MB. As shown in Figure 16, Prefill and Decode kernels account for 87.9% of its footprint, with Prefill alone contributing 70.9%. Prefill kernels dominate because they have high computational cost, complex implementations, and numerous schedule-parameter combinations that are enumerated and pre-specialized in AOT shared libraries.

As shown in Figure 17, the retained per-model libraries occupy 24.3–60.7 MB, reducing the universal archive by 88.8–95.5%. Notably, the experiment is conducted without cross-service deduplication, and the extrema place the storage break-even point at roughly 8–22 model services. In the worst case, serving 8 models like Mamba would require 8×60.7 MB = 485.6 MB, which is still smaller than the universal archive of 542 MB. A universal archive therefore favors diverse multi-model deployments, whereas per-model pruning favors serving modest model sets.

The reduction in library footprint benefits cold-start-sensitive scenarios such as serverless ML inference, where loading large operator libraries into GPU memory constitutes a major portion of initialization latency. Other DSA architectures with AoT-compiled operator libraries that suffer from similar code-bloat issues could also benefit from TilingInfer.

TilingInfer removes most of the binary footprint from Prefill and Decode kernels because these kernels enumerate and pre-specialize many combinations of template schedule parameters. After pruning these kernels, the retained libraries consist mainly of various complex utility kernels and SSU kernels.

5 Related Work

5.1 Partial Evaluation and Interprocedural Specialization

Constant propagation, partial evaluation, and application specialization are established compiler techniques. Sparse conditional constant propagation combines value and executable-edge reasoning [38], while systems such as TRIMMER specialize and debloat programs from deployment information [28]. C++ templates can also be understood as a manual form of partial evaluation [37]. TilingInfer builds on these foundations rather than claiming a new constant-propagation primitive. Its contribution is to recover a typed C++ entry state that is present in a framework Graph IR but absent from the operator library's LLVM IR, propagate that state through pointer-rich host code, and preserve precision across shape-checker-induced early-exit paths.

5.2 AI Compilers and Kernel Generation

~~IR-level schedule search for dynamic tensors. DietCode~~ [43] ~~introduces dynamic shape-aware auto-scheduling with symbolic tiling parameters. Nimble~~ [30] ~~provides graph-level compilation for dynamic neural networks. HAOTuner~~ [21] ~~and MikPoly~~ [41] ~~employ micro-kernel-based approaches for dynamic shapes. These systems operate within compiler IRs (TVM, Relax~~ [17]~~) and focus on generating kernels from high-level tensor expressions. In contrast, TilingInfer propagates invariants across the Python/C++ boundary into existing hand-written operator libraries.~~

~~Memory and layout optimization for dynamic shapes. BladeDISC~~ [46] ~~introduces symbolic shape representation for dynamic shape fusion. CoRa~~ [11] ~~addresses ragged tensor computation with minimal padding. While these optimize memory efficiency, they do not address specialization of pre-compiled hand-written kernels.~~

Dynamic-shape AI compilers such as DietCode [43], Nimble [30], BladeDISC [46], Relax [17], and CoRa [11] transform compiler-visible tensor programs and decide how to lower dynamic dimensions. Kernel languages and scheduling systems such as Triton [34], TileLang [33], and TVM/AutoTVM [6] help operator developers generate kernels and search or autotune schedules for the target shapes. Once a schedule is selected, its schedule parameters are encoded directly in the generated implementation.

~~Industrial practices for dynamic shapes. PyTorch 2~~ [3] ~~introduces SymInt and ShapeEnv for symbolic shape reasoning. TensorRT~~ [23] ~~supports dynamic shapes through profile-based optimization. However, these frameworks have limited support for specializing third-party AoT libraries (FlashInfer~~ [40]~~, CANN~~ [4]~~).~~

LLM-assisted systems change how the target program is produced but retain this generation or translation paradigm. QiMeng-Xpiler combines LLM-guided transformations, symbolic repair, and hierarchical autotuning to translate tensor programs across accelerator programming interfaces [10]. KernelBench evaluates whether LLMs can generate correct and efficient GPU kernels from ML workloads [25]. These systems generate or translate an implementation and then optimize its transformation or schedule choices. They do not reconstruct deployment constants within an unchanged source-level expert implementation to partially evaluate scalar LLVM instructions or prune unreachable precompiled variants.

TilingInfer therefore has a different optimization object. It preserves the expert-written kernel, including its vendor-specific control flow, data movement, and intrinsic calls, and specializes standard scalar LLVM instructions such as loads, stores, arithmetic, branches, and address calculations. This distinction is methodological: comparing a generated kernel with an expert-written kernel would mix schedule and implementation quality with the effect of specialization that TilingInfer is designed to isolate.

6 Discussion

~~Limitations. First, the performance benefits vary across kernel types and hardware platforms. TilingInfer is most effective for lightweight kernels with high invocation frequency on Ascend, while on GPUs, further specialization on top of existing manual optimization yields limited improvement due to sufficient general-purpose registers and well-optimized baseline libraries.~~

Workload-dependent benefits. Our experiments show that the magnitude of TilingInfer's benefit depends strongly on workload characteristics. Recommendation and decode workloads benefit most because Attention and MatMul expose profitable invariant schedules, yielding about 10% average inference speedup. Schedule-fixed expert-written libraries also benefit from removing unreachable template instances. This per-model pruning is most attractive when a deployment serves fewer models, because each pruned binary can discard a larger fraction of the universal library. By contrast, HSTU and prefill remain compute-heavy, so specializing their core Attention and MatMul kernels removes little of the total work and improves end-to-end latency by only about 3%. Some kernels in the current open-source baselines also cannot be further specialized by TilingInfer, further limiting end-to-end gains.

~~Second, the current prototype is based on PyTorch; applying TilingInfer to other frameworks like TensorFlow requires constructing update rules for their specific data types.~~

Platform applicability. Our schedule-generic kernel-specialization implementation currently targets Ascend. In the open-source libraries we surveyed for other accelerators, expert-written kernels predominantly follow the schedule-fixed design, leaving no comparable schedule-generic baseline to evaluate. Ascend exposes many programmable hardware levels; achieving high performance therefore relies on numerous runtime schedule parameters, which motivates schedule-generic libraries and makes automatic specialization particularly relevant.

~~Third, while TilingInfer can potentially specialize Host-side programs, such optimization involves more complex control flows and external function calls, which we leave for future work.~~

Optimization scope and limitations. TilingInfer applies to existing expert-written operator libraries: it specializes schedule-generic kernels or removes unreachable instances from schedule-fixed libraries. This differs from AI compilers, which generate implementations and search schedules; their code-generation pipelines do not optimize an existing C++ expert library in place.

In addition, TilingInfer analyzes host code only to recover constants and launch-site liveness, then optimizes reachable device kernels; it does not rewrite host code. Host control flow is complex, CPUs already execute it efficiently, and profitable host specialization would require a more selective policy than device-kernel constant propagation. NPU Graph replay suppresses repeated launch overhead, while identical input shapes across Transformer layers allow host-computed schedule parameters to be reused. Host schedule computation is therefore not a bottleneck in our evaluated setting, making device-kernel optimization the higher-value target.

7 Conclusion

~~We present TilingInfer, a static analysis framework that enables precise constant propagation across language boundaries for kernel specialization in deep learning operator libraries. Experimental results demonstrate its effectiveness: on Ascend NPU, TilingInfer achieves an average speedup of 1.06× (up to 1.17×) for individual operators and 3% end-to-end inference speedup for LLMs; on GPU, TilingInfer reduces the binary size of the FlashInfer library by 96.2%, effectively addressing the code bloat problem.~~

We presented TilingInfer, which propagates model-specific Graph-IR constants through existing C++ operator libraries to specialize schedule-generic device kernels or prune unreachable schedule-fixed instances.

Across 14 Ascend operators, TilingInfer delivers a 1.06× geometric-mean device-kernel speedup, with a maximum of 1.17×; its largest end-to-end gains occur in recommendation and decode workloads whose Attention and MatMul schedules expose profitable invariants, averaging about 10%.

For the schedule-fixed FlashInfer library, per-model pruning reduces the 542 MB universal binary to 24.3–60.7 MB across 17 model configurations, an 88.8–95.5% reduction, demonstrating that graph-derived deployment knowledge can improve either device-kernel execution or binary footprint without synthesizing new schedules.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, et al. 2016. TensorFlow: A system for large-scale machine learning. In 12th OSDI. USENIX Association, USA, 265–283.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4895–4901. doi:10.18653/v1/2023.emnlp-main.298

[3] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, David Berard, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Laurent Kirsch, Michael Lazos, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366

[4] Ascend. 2024. cann-ops-adv: An Advanced Fusion Operator Library for Ascend Hardware. https://gitee.com/ascend/cann-ops-adv A library of fusion operators based on Ascend hardware..

[5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL] https://arxiv.org/abs/2004.05150

[6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: an automated end-to-end optimizing compiler for deep learning. In Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (Carlsbad, CA, USA) (OSDI'18). USENIX Association, USA, 579–594.

[7] NVIDIA Corporation. 2017. CUTLASS: CUDA Template Library for Linear Algebra Subroutines. https://github.com/NVIDIA/cutlass. Accessed: 2024-07-09.

[8] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, Vienna, Austria, 14 pages. https://openreview.net/forum?id=mZn2Xyh9Ec

[9] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[10] Shouyang Dong, Jun Bi, Di Huang, Jiaming Guo, Jianxing Xu, Ruibai Xu, Xinkai Song, Yifan Hao, Ling Li, Xuehai Zhou, Tianshi Chen, Qi Guo, and Yunji Chen. 2025. QiMeng-Xpiler: Transcompiling Tensor Programs for Deep Learning Systems with a Neural-Symbolic Approach. In 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25). USENIX Association, Boston, MA, USA, 239–255. https://www.usenix.org/conference/osdi25/presentation/dong

[11] Pratik Fegade, Tianqi Chen, Phillip Gibbons, and Todd Mowry. 2022. The CoRa tensor compiler: Compilation for ragged tensors with minimal padding. Proceedings of Machine Learning and Systems 4 (2022), 721–747.

[12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine Based Neural Network for CTR Prediction. In Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence. International Joint Conferences on Artificial Intelligence, Melbourne, Australia, 1725–1731. doi:10.24963/ijcai.2017/239

[13] Shengyu Liu Jiashi Li. 2025. FlashMLA: Efficient Multi-head Latent Attention Kernels. https://github.com/deepseek-ai/FlashMLA.

[14] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In Proceedings of the 50th annual international symposium on computer architecture. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3579371.3589350

[15] Shuangxiang Kan, Yuhao Gao, Zexin Zhong, and Yulei Sui. 2024. Cross-Language Taint Analysis: Generating Caller-Sensitive Native Code Specification for Java. IEEE Trans. Softw. Eng. 50, 6 (June 2024), 1518–1533. doi:10.1109/TSE.2024.3392254

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[17] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, et al. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. Association for Computing Machinery, New York, NY, USA, 998–1013. doi:10.1145/3676641.3716249

[18] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization (Palo Alto, California) (CGO '04). IEEE Computer Society, USA, 75.

[19] Heng Liao, Jiajin Tu, Jing Xia, and Xiping Zhou. 2019. Davinci: A scalable architecture for neural network computing. In 2019 IEEE Hot Chips 31 Symposium (HCS). IEEE Computer Society, IEEE, Cupertino, CA, USA, 1–44.

[20] Raphaël Monat, Abdelraouf Ouadjaout, and Antoine Miné. 2021. A Multilanguage Static Analysis of Python Programs with Native C Extensions. In Static Analysis: 28th International Symposium, SAS 2021, Chicago, IL, USA, October 17–19, 2021, Proceedings (Chicago, IL, USA). Springer-Verlag, Berlin, Heidelberg, 323–345. doi:10.1007/978-3-030-88806-0_16

[21] Pengyu Mu, Yi Liu, Rui Wang, Guoxiang Liu, Zhonghao Sun, Hailong Yang, Zhongzhi Luan, and Depei Qian. 2023. HAOTuner: A Hardware Adaptive Operator Auto-Tuner for Dynamic Shape Tensor Compilers. IEEE Trans. Comput. 72, 11 (2023), 3191–3204.

[22] NVIDIA. 2021. cuBLAS library (11.2). Online at https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf. https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf

[23] NVIDIA. 2021. TensorRT Developer Guide. https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html.

[24] OpenXLA Project. 2024. StableHLO: High Level Operations for Machine Learning. https://openxla.org/stablehlo. Accessed: 2026-05-29.

[25] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs.LG] https://arxiv.org/abs/2502.10517

[26] Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In 32th NeurIPS. Curran Associates, Inc., Conference Location Unknown, 8024–8035.

[27] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: A New IR for Machine Learning Frameworks. In 2nd MAPL (Philadelphia, PA, USA). Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3211346.3211348

[28] Hashim Sharif, Muhammad Abubakar, Ashish Gehani, and Fareed Zaffar. 2018. TRIMMER: Application Specialization for Code Debloating. In Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. ACM, Montpellier France, 329–339. doi:10.1145/3238147.3238160

[29] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150 [cs.NE] https://arxiv.org/abs/1911.02150

[30] Haichen Shen, Jared Roesch, Zhi Chen, et al. 2021. Nimble: Efficiently compiling dynamic neural networks for model inference. In 4th MLSys, Vol. 3. Publisher Unknown, Conference Location Unknown, 208–222.

[31] Tensorflow. 2021. XLA: Optimizing Compiler for Machine Learning. https://www.tensorflow.org/xla.

[32] Chen Tianqi, Eddie Yan, et al. 2018. Learning to Optimize Tensor Programs. In 32nd NeurIPS. Publisher Unknown, Conference Location Unknown, 3393–3404.

[33] TileLang Team. 2024. TileLang: A Flexible and High-Performance DSL for Kernel Development. https://github.com/tile-ai/tilelang. Accessed: 2024.

[34] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In 3rd MAPL (Phoenix, AZ, USA) (MAPL 2019). Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[35] Hugo Touvron and Louis Martin. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems 30, "" (2017), "". No formal volume, number, or pages available.

[37] Todd L. Veldhuizen. 1998. C++ Templates as Partial Evaluation. arXiv:cs/9810010 [cs.PL] https://arxiv.org/abs/cs/9810010

[38] Mark N. Wegman and F. Kenneth Zadeck. 1991. Constant Propagation with Conditional Branches. ACM Trans. Program. Lang. Syst. 13, 2 (April 1991), 181–210. doi:10.1145/103135.103136

[39] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. arXiv:2407.10671 [cs.CL] https://arxiv.org/abs/2407.10671

[40] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In Proceedings of Machine Learning and Systems, Vol. 7. MLSys, Santa Clara, CA, USA, 17 pages. https://proceedings.mlsys.org/paper_files/paper/2025/file/dbf02b21d77409a2db30e56866a8ab3a-

Paper-Conference.pdf

[41] Feng Yu, Guangli Li, Jiacheng Zhao, Huimin Cui, Xiaobing Feng, and Jingling Xue. 2024. Optimizing Dynamic-Shape Neural Networks on Accelerators via On-the-Fly Micro-Kernel Polymerization. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 797–812. doi:10.1145/3620665.3640390

[42] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235). PMLR, Vienna, Austria, 58484–58509. https://proceedings.mlr.press/v235/zhai24a.html

[43] Bojian Zheng, Ziheng Jiang, Cody Hao Yu, Haichen Shen, Joshua Fromm, Yizhi Liu, Yida Wang, Luis Ceze, Tianqi Chen, and Gennady Pekhimenko. 2022. DietCode: Automatic optimization for dynamic tensor programs. Proceedings of Machine Learning and Systems 4 (2022), 848–863.

[44] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, Conference Location Unknown, 863–879. https://www.usenix.org/conference/osdi20/presentation/zheng

[45] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. Advances in neural information processing systems 37 (2024), 62557–62583.

[46] Zhen Zheng, Zaifeng Pan, Dalin Wang, Kai Zhu, Wenyi Zhao, Tianyou Guo, Xiafei Qiu, Minmin Sun, Junjie Bai, Feng Zhang, Xiaoyong Du, Jidong Zhai, and Wei Lin. 2023. BladeDISC: Optimizing Dynamic Shape Machine Learning Workloads via Compiler Approach. Proc. ACM Manag. Data 1, 3 (nov 2023), 1–29. doi:10.1145/3617327